

Chapter 2

Introduction to HTML and XHTML

Origins and Evolution of HTML

- HTML is defined with the use of the Standard Generalized Markup Language (SGML), which is an International Standards Organization (ISO) standard notation for describing text-formatting languages.
- Original **intent** of HTML: *General layout of documents that could be displayed by a wide variety of computers using different browsers.*
- In late 1994, Tim Berners-Lee, developed the initial version of HTML.

Other HTML Versions:

Version	Year (A.D.)
HTML 2.0	1995
HTML 3.2/ HTML 4.0	1997
HTML 4.0.1	1999
XHTML 1.0 (Just defined HTML 4.0.1 using XML instead of SGML; Developed standards: strict, transitional and Frameset)	2000
XHTML 1.1 (Modularized 1.0 and dropped frames)	2001
HTML5	2014

What is HTML?

HTML is the standard markup language for creating Web pages.

- HTML stands for Hyper Text Markup Language
- HTML describes the structure of Web pages using markup
- HTML elements are the building blocks of HTML pages
- HTML elements are represented by tags
- HTML tags label pieces of content such as "heading", "paragraph", "table", and so on
- Browsers do not display the HTML tags, but use them to render the content of the page

A Simple HTML Document

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

Where,

The `<!DOCTYPE html>` declaration defines this document to be HTML5

The `<html>` element is the root element of an HTML page

The `<head>` element contains meta information about the document

The `<title>` element specifies a title for the document

The `<body>` element contains the visible page content

The `<h1>` element defines a large heading

The `<p>` element defines a paragraph

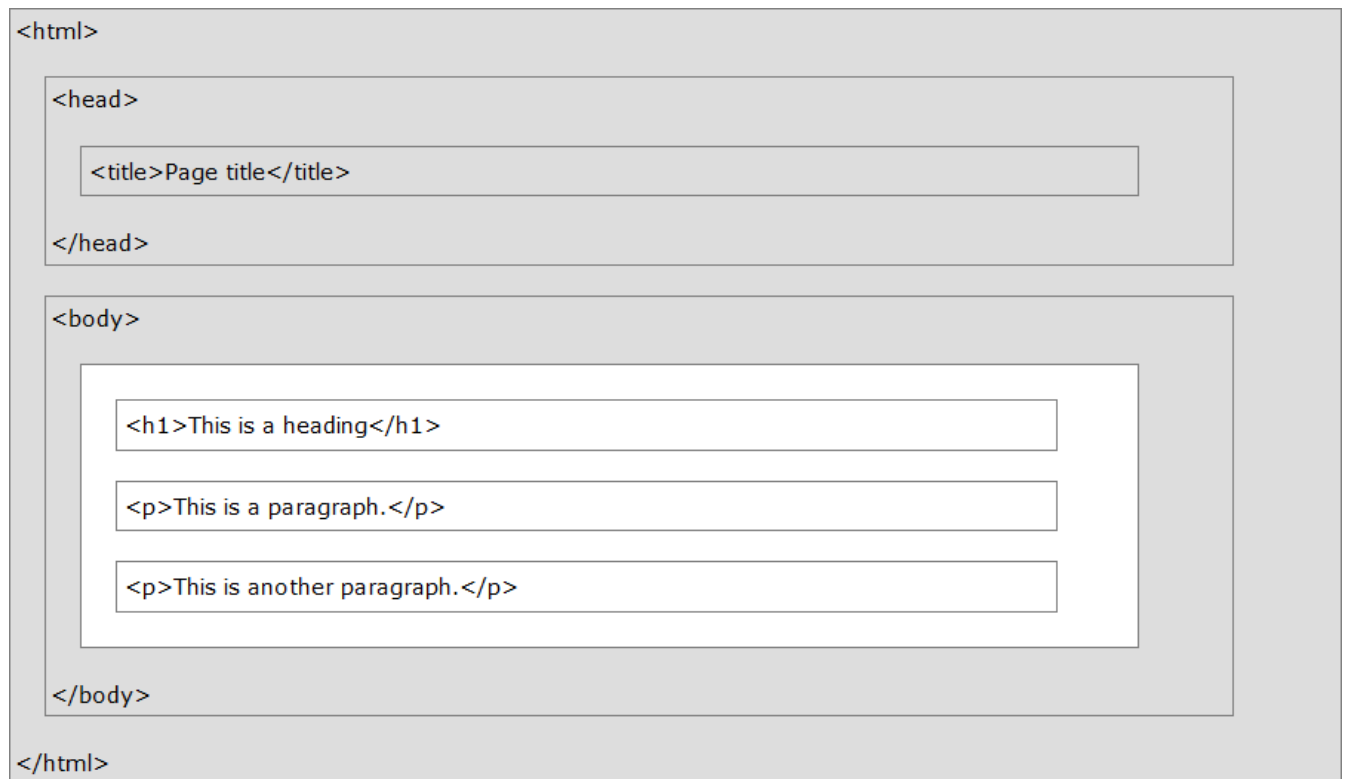
Basic Syntax:

- The fundamental syntactic units of HTML are called **tags**.
- In general, tags are used to specify categories of content.
- For each category, a browser has default presentation specifications for the specified content.
- HTML tags are element names surrounded by angle brackets:
 `<tagname> content goes here...</tagname>`
- HTML tags normally come **in pairs** like `<p>` and `</p>`
- The first tag in a pair is the **start tag**, the second tag is the **end tag**

- The end tag is written like the start tag, but with a **forward slash** inserted before the tag name
- Comments can appear in HTML/ XHTML in the following form:
<!-- anything except two adjacent dashes -->
- Browsers ignore HTML/XHTML comments—they are for users only.

HTML Page Structure

Below is a visualization of an HTML page structure:



Only the content inside the <body> section (the white area above) is displayed in a browser.

The <!DOCTYPE> Declaration

The <!DOCTYPE> declaration represents the document type, and helps browsers to display web pages correctly.

It must only appear once, at the top of the page (before any HTML tags).

The <!DOCTYPE> declaration is not case sensitive.

The <!DOCTYPE> declaration for HTML5 is:

```
<!DOCTYPE html>
```

HTML Elements

An HTML element usually consists of a **start** tag and **end** tag, with the content inserted in between:

<tagname>Content goes here...</tagname>

The HTML **element** is everything from the start tag to the end tag:

`<p>my first paragraph. </p>`

Start tag	Element content	End tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>	Line Break	

Nested HTML Elements

HTML elements can be nested (elements can contain elements).

All HTML documents consist of nested HTML elements.

Empty HTML Elements

HTML elements with no content are called empty elements.

`<br>` is an empty element without a closing tag (the `<br>` tag defines a line break).

Empty elements can be "closed" in the opening tag like this: `<br />`.

HTML5 does not require empty elements to be closed. But if you want stricter validation, or if you need to make your document readable by XML parsers, you must close all HTML elements properly.

HTML Attributes

- All HTML elements can have **attributes**
- Attributes provide **additional information** about an element
- Attributes are always specified in **the start tag**
- Attributes usually come in name/value pairs like: **name="value"**

The href Attribute

HTML links are defined with the `<a>` tag. The link address is specified in the **href** attribute:

Example

```
<a href="https://www.w3schools.com">This is a link</a>
```

The src Attribute

HTML images are defined with the `<img>` tag.

The filename of the image source is specified in the **src** attribute:

Example

```

```

The width and height Attributes

Images in HTML have a set of size attributes, which specifies the width and height of the image:

Example

```

```

The alt Attribute

The **alt** attribute specifies an alternative text to be used, when an image cannot be displayed.

The value of the attribute can be read by screen readers. This way, someone "listening" to the webpage, e.g. a blind person, can "hear" the element.

Example

```

```

The style Attribute

The style attribute is used to specify the styling of an element, like color, font, size etc.

Example

```
<p style="color:red">I am a paragraph</p>
```

The lang Attribute

The language of the document can be declared in the `<html>` tag.

The language is declared with the **lang** attribute.

Declaring a language is important for accessibility applications (screen readers) and search engines:

```
<!DOCTYPE html>  
<html lang="en-US">  
<body>
```

...

```
</body>  
</html>
```

The title Attribute

Here, a **title** attribute is added to the `<p>` element. The value of the title attribute will be displayed as a tooltip when you mouse over the paragraph:

Example

```
<p title="I'm a tooltip">  
This is a paragraph.  
</p>
```

HTML Headings

Headings are defined with the `<h1>` to `<h6>` tags.

`<h1>` defines the most important heading. `<h6>` defines the least important heading.

Example

```
<h1>This is heading 1</h1>  
<h2>This is heading 2</h2>  
<h3>This is heading 3</h3>
```

Search engines use the headings to index the structure and content of your web pages.

HTML Horizontal Rules

The `<hr>` tag defines a thematic break in an HTML page, and is most often displayed as a horizontal rule.

The `<hr>` element is used to separate content (or define a change) in an HTML page:

Example

```
<h1>This is heading 1</h1>
<p>This is some text.</p>
<hr>
<h2>This is heading 2</h2>
<p>This is some other text.</p>
<hr>
```

The HTML `<head>` Element

The HTML `<head>` element has nothing to do with HTML headings.

The `<head>` element is a container for metadata. HTML metadata is data about the HTML document. Metadata is not displayed.

The `<head>` element is placed between the `<html>` tag and the `<body>` tag:

Example

```
<!DOCTYPE html>
<html>

<head>
  <title>My First HTML</title>
  <meta charset="UTF-8">
</head>
<body>
```

HTML Paragraphs

HTML paragraphs are defined with the `<p>` tag:

Example

```
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>
```

The HTML `<pre>` Element

The HTML `<pre>` element defines preformatted text.

The text inside a `<pre>` element is displayed in a fixed-width font (usually Courier), and it preserves both spaces and line breaks:

Example

```
<pre>
    My Bonnie lies over the ocean.
    My Bonnie lies over the sea.

    My Bonnie lies over the ocean.
    Oh, bring back my Bonnie to me.
</pre>
```

The HTML Style Attribute

Setting the style of an HTML element, can be done with the **style attribute**.

The HTML style attribute has the following **syntax**:

```
<tagname style="property:value;">
```

The **property** is a CSS property. The **value** is a CSS value.

HTML Background Color

The **background-color** property defines the background color for an HTML element.

This example sets the background color for a page to powderblue:

Example

```
<body style="background-color:blue;">
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
```

HTML Text Color

The **color** property defines the text color for an HTML element:

Example

```
<h1 style="color:blue;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
```

HTML Text Alignment

The **text-align** property defines the horizontal text alignment for an HTML element:

Example

```
<h1 style="text-align:center;">Centered Heading</h1>
<p style="text-align:center;">Centered paragraph.</p>
```

HTML Text Formatting

HTML Formatting Elements

HTML uses elements like `<b>` and `<i>` for formatting output, like **bold** or *italic* text. Formatting elements were designed to display special types of text:

- `<b>` - Bold text
- `<strong>` - Important text
- `<i>` - Italic text
- `<em>` - Emphasized text
- `<mark>` - Marked text
- `<small>` - Small text
- `<del>` - Deleted text
- `<ins>` - Inserted text
- `<sub>` - Subscript text
- `<sup>` - Superscript text

HTML `<b>` and `<strong>` Elements

The HTML `<b>` element defines **bold** text, without any extra importance.

Example

```
<b>This text is bold</b>
```

The HTML `<strong>` element defines **strong** text, with added semantic "strong" importance.

Example

```
<strong>This text is strong</strong>
```

- Subscript and superscript characters can be specified by the `<sub>` and `<sup>` tags, respectively. These are not content-based tags. For example:
 $X_{\text{sub}2}^{\text{sup}3} + y_{\text{sub}1}^{\text{sup}2}$

$$x_2^3 + y_1^2$$

Character Entities

- HTML/XHTML provides a collection of special characters that are sometimes needed in a document but cannot be typed as themselves.
- In some cases, these characters are used in XHTML in some special way—for example, `>`, `<`, and `&`.
- In other cases, the characters do not appear on keyboards, such as the small raised circle that represents “degrees” in a reference to temperature.

- Finally, there is the nonbreaking space, which browsers regard as a hard space—they do not squeeze them out, as they do other multiple spaces.
- These special characters are defined as entities, which are codes for the characters. An entity in a document is replaced by its associated character by the browser

Table 2.1 Some commonly used entities

Character	Entity	Meaning
&	<code>&amp;</code>	Ampersand
<	<code>&lt;</code>	Is less than
>	<code>&gt;</code>	Is greater than
"	<code>&quot;</code>	Double quote
'	<code>&apos;</code>	Single quote (apostrophe)
$\frac{1}{4}$	<code>&frac14;</code>	One-quarter
$\frac{1}{2}$	<code>&frac12;</code>	One-half
$\frac{3}{4}$	<code>&frac34;</code>	Three-quarters
°	<code>&deg;</code>	Degree
(space)	<code>&nbsp;</code>	Nonbreaking space

HTML Quotation and Citation Elements

The HTML `<q>` element defines a short quotation.

Browsers usually insert quotation marks around the `<q>` element.

Example

```
<p>WWF's goal is to: <q>Build a future where people live in harmony with nature.</q></p>
```

HTML `<blockquote>` for Quotations

The HTML `<blockquote>` element defines a section that is quoted from another source.

Browsers usually indent `<blockquote>` elements.

Example

```
<p>Here is a quote from WWF's website:</p>
<blockquote cite="http://www.worldwildlife.org/who/index.html">
For 50 years, WWF has been protecting the future of nature.
The world's leading conservation organization,
WWF works in 100 countries and is supported by
1.2 million members in the United States and
close to 5 million globally.
</blockquote>
```

HTML **<abbr>** for Abbreviations

The HTML **<abbr>** element defines an abbreviation or an acronym.

Marking abbreviations can give useful information to browsers, translation systems and search-engines.

Example

```
<p>The <abbr title="World Health Organization">WHO</abbr> was founded in 1948.</p>
```

HTML **<address>** for Contact Information

The HTML **<address>** element defines contact information (author/owner) of a document or an article.

The **<address>** element is usually displayed in italic. Most browsers will add a line break before and after the element.

Example

```
<address>
Written by John Doe.<br>
Visit us at:<br>
Example.com<br>
Box 564, Disneyland<br>
USA
</address>
```

HTML **<cite>** for Work Title

The HTML **<cite>** element defines the title of a work.

Browsers usually display **<cite>** elements in italic.

Example

```
<p><cite>The Scream</cite> by Edvard Munch. Painted in 1893.</p>
```

HTML <bdo> for Bi-Directional Override

The HTML <bdo> element defines bi-directional override.

The <bdo> element is used to override the current text direction:

Example

```
<bdo dir="rtl">This text will be written from right to left</bdo>
```

The meta Element

- The meta element is used to provide additional information about a document.
- The meta elements are specified with <meta> tags and has no content; rather, all of the information provided is specified with attributes.
- The two attributes that are used to provide information are **name** and **content**.
- The user makes up a name as the value of the name attribute and specifies information through the content attribute.
- One commonly chosen name is **keywords**; the value of the content attribute associated with the keywords are those which the author of a document believes characterizes his or her document.
- An example is

```
<meta name = "keywords" content = "binary trees, linked lists, stacks" />
```
- Web search engines use the information provided with the meta element to categorize Web documents in their indices.
- So, if the author of a document seeks widespread exposure for the document, one or more meta elements are included to ensure that it will be found by Web search engines. For example, if an entire book were published as a Web document, it might have the following meta elements:

```
<meta name = "Title" content = "Don Quixote" />  
<meta name = "Author" content = "Miguel Cervantes" />  
<meta name = "keywords" content = "novel,  
Spanish literature, groundbreaking work" />
```

Common Image Formats and Comparison:

- The most common methods of representing images in Web are Graphic Interchange Format (GIF), Joint Photographic Experts Group (JPEG) and Portable Network Graphics (PNG).
- GIF images use .gif (or .GIF) extension, JPEG images use .jpg (or .JPG or .jpeg) extension while PNG images use .png (or .PNG) extension in their names.
- **Color representation:** GIF images use 8-bit color representation for pixels allowing a pixel to have 256 different colors. The JPEG format uses 24-bit color representations for pixels which allows JPEG images to include more than 16 million colors. While PNG

images use at least 24-bit color representation or widely 32-bit color representation (RGB+alpha) for pixel. Thus, PNG is the best one in terms of color representation.

- **Transparency:** GIF and PNG images can be made to appear transparent but JPEG images donot support transparency.
- **Compression and Space:** The compression algorithm used by JPEG is better than that used by GIF and PNG images. Thus, for same object JPEG image representation require much less space than GIF and PNG images. Thus, JPEG images are often preferred to GIF and PNG images in terms of space.

HTML Links - Hyperlinks

HTML links are hyperlinks.

Clicking on a link we can jump to another document.

When you move the mouse over a link, the mouse arrow will turn into a little hand.

Note: A link does not have to be text. It can be an image or any other HTML element.

HTML Links - Syntax

In HTML, links are defined with the **<a>** tag:

```
<a href="url">link text</a>
```

The **href** attribute specifies the destination address (<https://www.w3schools.com/html/>) of the link. The **link text** is the visible part

Local Links

A local link (link to the same web site) is specified with a relative URL (without <http://www...>).

Example

```
<a href="html_images.asp">HTML Images</a>
```

HTML Link Colors

By default, a link will appear like this (in all browsers):

- An unvisited link is underlined and blue
- A visited link is underlined and purple
- An active link is underlined and red

You can change the default colors, by using styles:

Example

```
<style>
a:link {
    color: green;
    background-color: transparent;
    text-decoration: none;
}

a:visited {
    color: pink;
    background-color: transparent;
    text-decoration: none;
}

a:hover {
    color: red;
    background-color: transparent;
    text-decoration: underline;
}

a:active {
    color: yellow;
    background-color: transparent;
    text-decoration: underline;
}
</style>
```

HTML Links - The target Attribute

The **target** attribute specifies where to open the linked document.

The target attribute can have one of the following values:

- `_blank` - Opens the linked document in a new window or tab
- `_self` - Opens the linked document in the same window/tab as it was clicked (this is default)
- `_parent` - Opens the linked document in the parent frame
- `_top` - Opens the linked document in the full body of the window
- `framename` - Opens the linked document in a named frame

This example will open the linked document in a new browser window/tab:

Example

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

HTML Links - Image as Link

It is common to use images as links:

Example

```
<a href="default.asp">  
    
</a>
```

HTML Images

Images can improve the design and the appearance of a web page.

Example

```

```

HTML Images Syntax

In HTML, images are defined with the **** tag.

The **** tag is empty, it contains attributes only, and does not have a closing tag.

The src attribute specifies the URL (web address) of the image:

```

```

The alt Attribute

The alt attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the src attribute, or if the user uses a screen reader).

The value of the alt attribute should describe the image:

If a browser cannot find an image, it will display the value of the alt attribute:

Images in Another Folder

If not specified, the browser expects to find the image in the same folder as the web page.

However, it is common to store images in a sub-folder. You must then include the folder name in the src attribute:

Example

```

```

Images on another Server

Some web sites store their images on image servers.

Actually, you can access images from any web address in the world:

Example

```

```

Animated Images

HTML allows animated GIFs:

Example

```

```

Image as a Link

To use an image as a link, put the `<img>` tag inside the `<a>` tag:

Example

```
<a href="default.asp">  
    
</a>
```

Image Floating

Use the CSS **float** property to let the image float to the right or to the left of a text:

Example

```
<p>
```

The image will float to the right of the text.</p>

```
<p>
```

The image will float to the left of the text.</p>

Image Maps

Use the `<map>` tag to define an image-map. An image-map is an image with clickable areas.

Example

```
  
<map name="workmap">  
  <area shape="rect" coords="34,44,270,350" alt="Computer" href="computer.htm">  
  <area shape="rect" coords="290,172,333,250" alt="Phone" href="phone.htm">  
  <area shape="circle" coords="337,300,44" alt="Coffee" href="coffee.htm">  
</map>
```

The name attribute of the `<map>` tag is associated with the `<img>`'s `usemap` attribute and creates a relationship between the image and the map.

The `<map>` tag contains a number of `<area>` tags, which defines the clickable areas in the image-map.

The `<area>` tag defines an area inside an image-map (an image-map is an image with clickable areas). The `<area>` element is always nested inside a `<map>` tag.

The **coords** attribute specifies the coordinates of an area in an image-map.

x1,y1,x2,y2 : Specifies the coordinates of the top-left and bottom-right corner of the rectangle (shape="rect")

x,y,radius: Specifies the coordinates of the circle center and the radius (shape="circle")

Background Image

To add a background image on an HTML element, use the CSS property background-image:

Example

To add a background image on a web page, specify the background-image property on the BODY element:

```
<body style="background-image:url('clouds.jpg')">
<h2>Background Image</h2>
</body>
```

Example

To add a background image on a paragraph, specify the background-image property on the P element:

```
<body>
<p style="background-image:url('clouds.jpg')">
...
</p>
</body>
```

The `<picture>` Element

HTML5 introduced the `<picture>` element to add more flexibility when specifying image resources.

The `<picture>` element contains a number of `<source>` elements, each referring to different image sources. This way the browser can choose the image that best fit the current view and/or device.

Each `<source>` element have attributes describing when their image is the most suitable.

The browser will use the first <source> element with matching attribute values, and ignore any following <source> elements.

Example

Show one picture if the browser window (viewport) is a minimum of 650 pixels, and another image if not, but larger than 465 pixels.

```
<picture>
<source media="(min-width: 650px)" srcset="img_pink_flowers.jpg">
<source media="(min-width: 465px)" srcset="img_white_flower.jpg">

</picture>
```

HTML Tables

A table is a matrix of rows and columns, in which each intersection of a row and column is called a **cell**.

An HTML table is defined with the <**table**> tag.

Each table row is defined with the <**tr**> tag. A table header is defined with the <**th**> tag. By default, table headings are bold and centered. A table data/cell is defined with the <**td**> tag. Example:

```
<table>
  <tr>  <th>Firstname</th>  <th>Lastname</th>  <th>Age</th>  </tr>
  <tr>  <td>Jill</td>  <td>Smith</td>  <td>50</td>  </tr>
  <tr>  <td>Eve</td>  <td>Jackson</td>  <td>94</td>  </tr>
</table>
```

Output:

Firstname	Lastname	Age
Jill	Smith	50
Eve	Jackson	94
John	Doe	80

HTML Table - Adding a Border

If you do not specify a border for the table, it will be displayed without borders.

We can specify border using **border** attribute of a table and giving specific value as per the strength (boldness) requirement of the border. E.g. <table border="1">

OR

A border can be set using the CSS **border** property:

Example

```
table, th, td {  
    border: 1px solid black;  
}
```

HTML Table - Collapsed Borders

By default, **table** **td** provides border to each cell separately, which may be useful sometimes but most often we need separate border to be collapsed to single one.

If you want the borders to collapse into one border, we can give **cellspacing="0"** attribute of `<table>` tag or we add the CSS **border-collapse** property using style attribute:

Example

```
<table cellspacing="0">
```

OR

```
<table style="border-collapse:collapse">
```

HTML Table - Adding Cell Padding

Cell padding specifies the space between the cell content and its borders.

If you do not specify a padding, the table cells will be displayed without padding.

To set the padding, we can use table attribute – **cellpadding** that provides padding to each cell from all direction (i.e. top, right, bottom, and left). E.g. `<table cellpadding="5">`

OR

We can use the CSS **padding** property as well:

Example

```
th, td {  
    padding: 15px;  
}
```

HTML Table -width attribute

If the width attribute is not set, the `<table>` takes up the space it needs to display the table data.

But, we can specify the width of whole table or the cells using **width** attribute on `<table>` tag or `<td>` or `<th>` tag. We can provide value of width on **pixels format or percentage (%) format**.

Example:

`<table width="500">`, takes up 500px width for the table

`<td width="50 %">`, takes up 50% of total table width for this cell.

Note: **width** attribute is not supported in HTML5, so we instead use width property of CSS style attribute
E.g. `<table style="width: 100 %">`, specifies the table to take all available space for displaying table.

HTML Table -align attribute

We can use **align** and **valign** (for vertical align) attribute of table to specify the alignment of texts within the table cell.

We can specify align and valign attributes on `<table>` or `<tr>` or `<td>` tags, but most commonly we specify it on `<tr>` tag so that it applies on all cells of that row.

Example:

`<tr align="center">`

There can be **left**, **center** and **right** values for **align** attribute while there can be **top**, **center** and **bottom** values for valign attribute.

Note: align & valign properties are also not supported in HTML5, so we use CSS style attribute and text-align property.

By default, table headings are bold and centered.

To left-align the table headings, use the CSS **text-align** property:

Example

```
th {  
    text-align: left;  
}
```

HTML Table - Adding Border Spacing

Border spacing specifies the space between the cells.

To set the border spacing for a table, we can use **cellspacing** attribute on `<table>` tag.

Example: `<table cellspacing="5">`

We can also use the CSS **border-spacing** property:

```
table {
  border-spacing: 5px;
}
```

HTML Table - Cells that Span Many Columns

To make a cell span more than one column, use the **colspan** attribute:

Example

```
<table>
  <tr>
    <th>Name</th>
    <th colspan="2">Telephone</th>
  </tr>
  <tr>
    <td>Bill Gates</td>
    <td>55577854</td>
    <td>55577855</td>
  </tr>
</table>
```

Name	Telephone	
Bill Gates	55577854	55577855

HTML Table - Cells that Span Many Rows

To make a cell span more than one row, use the **rowspan** attribute:

Example

```
<table >
  <tr>
    <th>Name:</th>
    <td>Bill Gates</td>
  </tr>
  <tr>
    <th rowspan="2">Telephone:</th>
    <td>55577854</td>
  </tr>
  <tr>
    <td>55577855</td>
  </tr>
</table>
```

Name:	Bill Gates
Telephone:	55577854
	55577855

A Special Style for One Table

To define a special style for a special table, add an **id** attribute to the table:

```

<table id="t01">
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>94</td>
  </tr>
</table>

```

Now you can define a special style for this table:

```

table#t01 {
  width: 100%;
  background-color: #f1f1c1;
}

```

And add more styles:

```

table#t01 tr:nth-child(even) {
  background-color: #eee;
}
table#t01 tr:nth-child(odd) {
  background-color: #fff;
}
table#t01 th {
  color: white;
  background-color: black;
}

```

Summary

- Use the HTML **<table>** element to define a table
- Use the HTML **<tr>** element to define a table row
- Use the HTML **<td>** element to define a table data
- Use the HTML **<th>** element to define a table heading
- Use the HTML **<caption>** element to define a table caption
- Use the CSS **border** property to define a border
- Use the CSS **border-collapse** property to collapse cell borders
- Use the CSS **padding** property to add padding to cells
- Use the CSS **text-align** property to align cell text
- Use the CSS **border-spacing** property to set the spacing between cells
- Use the **colspan** attribute to make a cell span many columns
- Use the **rowspan** attribute to make a cell span many rows
- Use the **id** attribute to uniquely define one table

HTML Table Tags

Tag	Description
<u><table></u>	Defines a table
<u><th></u>	Defines a header cell in a table
<u><tr></u>	Defines a row in a table
<u><td></u>	Defines a cell in a table
<u><caption></u>	Defines a table caption
<u><colgroup></u>	Specifies a group of one or more columns in a table for formatting
<u><col></u>	Specifies column properties for each column within a <colgroup> element
<u><thead></u>	Groups the header content in a table
<u><tbody></u>	Groups the body content in a table
<u><tfoot></u>	Groups the footer content in a table

HTML Lists

HTML List Example

An Unordered List:

- Item
- Item
- Item
- Item

An Ordered List:

1. First item
2. Second item
3. Third item
4. Fourth item

Unordered HTML List

An unordered list starts with the **** tag. Each list item starts with the **** tag.

The list items will be marked with bullets (small black circles) by default:

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Unordered HTML List - Choose List Item Marker

The CSS **list-style-type** property is used to define the style of the list item marker:

Value	Description
disc	Sets the list item marker to a bullet (default)
circle	Sets the list item marker to a circle
square	Sets the list item marker to a square
none	The list items will not be marked

Example - Disc

```
<ul style="list-style-type:disc">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Example - Circle

```
<ul style="list-style-type:circle">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Example - Square

```
<ul style="list-style-type:square">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Example - None

```
<ul style="list-style-type:none">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Ordered HTML List

An ordered list starts with the **** tag. Each list item starts with the **** tag. The list items will be marked with numbers by default:

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Ordered HTML List - The Type Attribute

The **type** attribute of the **** tag, defines the type of the list item marker:

Type	Description
------	-------------

type="1"	The list items will be numbered with numbers (default)
----------	--

type="A"	The list items will be numbered with uppercase letters
----------	--

type="a"	The list items will be numbered with lowercase letters
----------	--

type="I"	The list items will be numbered with uppercase roman numbers
----------	--

type="i"	The list items will be numbered with lowercase roman numbers
----------	--

Numbers:

```
<ol type="1">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Uppercase Letters:

```
<ol type="A">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Lowercase Letters:

```
<ol type="a">
  <li>Coffee</li>
  <li>Tea</li> <li>Milk</li>
</ol>
```

Uppercase Roman Numbers:

```
<ol type="I">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Lowercase Roman Numbers:

```
<ol type="i">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

HTML Description Lists

HTML also supports description lists.

A description list is a list of terms, with a description of each term.

The **<dl>** tag defines the description list, the **<dt>** tag defines the term (name), and the **<dd>** tag describes each term:

Example

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- white cold drink</dd>
</dl>
```

o/p:

A Description List

Coffee

- black hot drink

Milk

- white cold drink

Nested HTML Lists

List can be nested (lists inside lists):

Example

```
<ul>
  <li>Coffee</li>
```

```

</li>Tea
  <ul>
    <li>Black tea</li>
    <li>Green tea</li>
  </ul>
</li>
<li>Milk</li>
</ul>

```

o/p:

- Coffee
- Tea
 - Black tea
 - Green tea
- Milk

Horizontal Lists

HTML lists can be styled in many different ways with CSS.

One popular way is to style a list horizontally, to create a menu:

Example

```

<!DOCTYPE html>
<html>
<head>
<style>
ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: #333333;
}

li {
  float: left;
}

li a {
  display: block;
  color: white;
  text-align: center;
  padding: 16px;
  text-decoration: none;
}

```

```

li a:hover {
    background-color: #111111;
}
</style>
</head>
<body>
<ul>
    <li><a href="#home">Home</a></li>
    <li><a href="#news">News</a></li>
    <li><a href="#contact">Contact</a></li>
    <li><a href="#about">About</a></li>
</ul>
</body>
</html>

```

Summary

- Use the HTML **** element to define an unordered list
- Use the CSS **list-style-type** property to define the list item marker
- Use the HTML **** element to define an ordered list
- Use the HTML **type** attribute to define the numbering type
- Use the HTML **** element to define a list item
- Use the HTML **<dl>** element to define a description list
- Use the HTML **<dt>** element to define the description term
- Use the HTML **<dd>** element to describe the term in a description list
- Lists can be nested inside lists
- List items can contain other HTML elements
- Use the CSS property **float:left** or **display:inline** to display a list horizontally

HTML Block and Inline Elements

Every HTML element has a default display value depending on what type of element it is. The default display value for most elements is block or inline.

Block-level Elements

A block-level element always starts on a new line and takes up the full width available (stretches out to the left and right as far as it can).

The `<div>` element is a block-level element.

Example

```

<div>Hello</div>
<div>World</div>

```

Inline Elements

An inline element does not start on a new line and only takes up as much width as necessary.

This is an inline element inside a paragraph.

Example

```
<span>Hello</span>
<span>World</span>
```

The <div> Element

The <div> element is often used as a container for other HTML elements.

The <div> element has no required attributes, but both **style** and **class** are common.

When used together with CSS, the <div> element can be used to style blocks of content:

Example

```
<div style="background-color:black;color:white;padding:20px;">
  <h2>London</h2>
  <p>London is the capital city of England. It is the most populous city in the United Kingdom, with a metropolitan area of over 13 million inhabitants.</p>
</div>
```

The Element

The element is often used as a container for some text.

The element has no required attributes, but both **style** and **class** are common.

When used together with CSS, the element can be used to style parts of the text:

Example

```
<h1>My <span style="color:red">Important</span> Heading</h1>
```

HTML Form

The <form> Element

The most common way for a user to communicate information from a Web Browser to the server is through the form.

HTML/XHTML provides tags to generate the most commonly used objects on a screen form. These objects are called **controls** or **widgets**.

There are controls for single-line and multiple-line text collection, checkboxes, radio buttons, and so on.

All control tags are inline tags.

Every form requires a **Submit** button.

The HTML **<form>** element defines a form that is used to collect user input:

```
<form>
.
form elements
.
</form>
```

An HTML form contains **form elements**.

Form elements are different types of input elements, like text fields, checkboxes, radio buttons, submit buttons, and more.

Attributes of <form> tag:

<form> tag has several different attributes. Some of the important attributes of <form> tag are as follows:

Attribute Name	Value(s)	Function
Action	url	Specifies the url of the processing page that is to be called when the user clicks the submit button.
Method	GET, POST	It is used to parse the form data submitted to the server. Default method is GET
Id	Idname	Specifies the id of the form
Name	Text	Specifies the name of the form.
Enctype	Application/x-www-form-urlencoded , multipart/form-data, text/plain	Specifies how the form data should be encoded while submitting it to the server. Used only for POST method.
Target	_blank,_self,_parent,_top	Specifies where to display response that is received after submitting the form.

HTML Form Elements

The <input> Element

The most important form element is the <input> element.

The <input> element can be displayed in several ways, depending on the **type** attribute.

Example

```
<input name="firstname" type="text">
```

Note: If the **type** attribute is omitted, the input field gets the default type: "text".

HTML Input Types

Input Type Text

<input type="text"> defines a **one-line text input field**.

A text control, which we usually refer to as **textbox** creates a horizontal box into which user can type text and thus we can gather information from the user.

The default size of textbox is often 20 characters.

Example

```
<form>  
  First name:<br>  
  <input type="text" name="firstname"><br>  
  Last name:<br>  
  <input type="text" name="lastname">  
</form>
```

This is how the HTML code above will be displayed in a browser:

First name:

Last name:

Input Type Password

<input type="password"> defines a **password field**:

```
User name:<br>
<input type="text" name="username"><br>
User password:<br>
<input type="password" name="psw">
</form>
```

Input Type Submit

<input type="submit"> defines a button for **submitting** form data to a **form-handler**.

The form-handler is typically a server page with a script for processing input data.

The form-handler is specified in the form's **action** attribute:

Example

```
<form action="/action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey"><br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse"><br><br>
  <input type="submit" value="Submit">
</form>
```

Input Type Reset

<input type="reset"> defines a **reset button** that will reset all form values to their default values:

Example

```
<form action="/action_page.php">
  First name:<br>
  <input type="text" name="firstname" value="Mickey"><br>
  Last name:<br>
  <input type="text" name="lastname" value="Mouse"><br><br>
  <input type="submit" value="Submit">
  <input type="reset">
</form>
```

Input Type Radio

<input type="radio"> defines a **radio button**.

Radio buttons let a user select ONLY ONE of a limited number of choices:

```
<input type="radio" name="gender" value="male" checked> Male<br>
<input type="radio" name="gender" value="female"> Female<br>
<input type="radio" name="gender" value="other"> Other
</form>
```

This is how the HTML code above will be displayed in a browser:

- ☒ Male
- ☐ Female
- ☐ Others

Input Type Checkbox

<input type="checkbox"> defines a **checkbox**.

Checkboxes let a user select ZERO or MORE options of a limited number of choices.

Example

```
<form>
  <input type="checkbox" name="vehicle1" value="Bike"> I have a bike<br>
  <input type="checkbox" name="vehicle2" value="Car"> I have a car
</form>
```

This is how the HTML code above will be displayed in a browser:

- ☐ I have a bike
- ☐ I have a car

Input type hidden

The **<input type="hidden">** defines a hidden input field.

The hidden field let web developers to include data that cannot be seen or modified by users when form is submitted.

Example:

```
<input type="hidden" name="studentId" value="1"/>
```

Input type file

The `<input type="file">` defines a file select field and a “Browse” button for file uploads.

To select multiple file, we can add attribute **multiple**.

To upload selected file to server while form submit, the “enctype” attribute of form must have a value “multipart/form-data”.

Example:

Select File: `<input type="file" name="myFile">`

Output:

Select File: No file chosen

Assignment:

- Explain the differences between usage of radio buttons, checkboxes and dropdown list in HTML form along with examples.

Input Type Button

`<input type="button">` defines a **button**:

Example

`<input type="button" onclick="alert('Hello World!')" value="Click Me!">`

HTML5 Input Types

HTML5 added several new input types:

- color
- date
- datetime-local
- email
- month
- number
- range
- search
- tel
- time
- url
- week

Input Type Color

The `<input type="color">` is used for input fields that should contain a color.

Depending on browser support, a color picker can show up in the input field.

Example

```
<form>
  Select your favorite color:
  <input type="color" name="favcolor">
</form>
```

Input Type Date

The `<input type="date">` is used for input fields that should contain a date.

Depending on browser support, a date picker can show up in the input field.

Example

```
1. <form>
  Birthday:
  <input type="date" name="bday">
</form>

2. <form>
  Enter a date before 1980-01-01:
  <input type="date" name="bday" max="1979-12-31"><br>
  Enter a date after 2000-01-01:
  <input type="date" name="bday" min="2000-01-02"><br>
</form>
```

Input Type Datetime-local

The `<input type="datetime-local">` specifies a date and time input field, with no time zone.

Depending on browser support, a date picker can show up in the input field.

Example

```
<form>
  Birthday (date and time):
  <input type="datetime-local" name="bdaytime">
</form>
```

Input Type Email

The `<input type="email">` is used for input fields that should contain an e-mail address.

Depending on browser support, the e-mail address can be automatically validated when submitted.

Some smartphones recognize the email type, and adds ".com" to the keyboard to match email input.

Example

```
<form>
```

E-mail:

```
<input type="email" name="email">
```

```
</form>
```

Input Restrictions

Here is a list of some common input restrictions (some are new in HTML5):

Attribute	Description
Disabled	Specifies that an input field should be disabled
Max	Specifies the maximum value for an input field
maxlength	Specifies the maximum number of character for an input field
Min	Specifies the minimum value for an input field
Pattern	Specifies a regular expression to check the input value against
Readonly	Specifies that an input field is read only (cannot be changed)
Required	Specifies that an input field is required (must be filled out)
Size	Specifies the width (in characters) of an input field
Step	Specifies the legal number intervals for an input field
Value	Specifies the default value for an input field
placeholder	Specifies a short hint that describes the expected value of input field.

The <select> Element

The `<select>` element defines a **drop-down list**:

Example

```
<select name="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

The **<option>** element defines an option that can be selected.

By default, the first item in the drop-down list is selected.

To define a pre-selected option, add the **selected** attribute to the option:

Example

```
<option value="fiat" selected>Fiat</option>
```

Visible Values:

Use the **size** attribute to specify the number of visible values:

Example

```
<select name="cars" size="3">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

Allow Multiple Selections:

Use the **multiple** attribute to allow the user to select more than one value:

Example

```
<select name="cars" size="4" multiple>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

The <textarea> Element

The **<textarea>** element defines a multi-line input field (a **text area**):

Example

```
<textarea name="message" rows="10" cols="30">
The cat was playing in the garden.
</textarea>
```

The **rows** attribute specifies the visible number of lines in a text area.

The **cols** attribute specifies the visible width of a text area.

The <button> Element

The <**button**> element defines a clickable **button**:

Example

```
<button type="button" onclick="alert('Hello World!')">Click Me!</button>
```

HTML5 Form Elements

HTML5 added the following form elements:

- <datalist>
- <output>

HTML5 <datalist> Element

The <**datalist**> element specifies a list of pre-defined options for an <input> element.

Users will see a drop-down list of the pre-defined options as they input data.

The **list** attribute of the <input> element, must refer to the **id** attribute of the <datalist> element.



Example

```
<form action="/action_page.php">
  <input list="browsers">
  <datalist id="browsers">
    <option value="Internet Explorer">
    <option value="Firefox">
    <option value="Chrome">
    <option value="Opera">
    <option value="Safari">
  </datalist>
</form>
```

HTML5 <output> Element

The <output> element represents the result of a calculation (like one performed by a script).



Example

Perform a calculation and show the result in an <output> element:

```
<form action="/action_page.php"
oninput="x.value=parseInt(a.value)+parseInt(b.value)">
0
<input type="range" id="a" name="a" value="50">
100 +
<input type="number" id="b" name="b" value="50">
=
<output name="x" for="a b"></output>
<br><br>
<input type="submit">
</form>
```

HTML Frames

Frames:

- The browser display window can be used to display more than one document at a time.
- The window can be divided into rectangular areas, each of which is a frame.
- Each frame is capable of displaying its own document.
- Frames can be used for a number of different display situations.
- Due to various reasons, use of frames was discouraged in XHTML 1.1, but currently **xframes** and **iframes** are used to develop a new form of Frames.
- The content of frame is specified with <frame> tag, which can appear only in the content of frameset element.

Framesets:

- The number of frames and their layouts in the browser window are specified with <frameset> tag.
- A frameset element takes the place of body element in the document.
- **A document can have either a body or a frameset but not both.**
- The <frameset> tags must have either a **rows** or a **cols** attribute, and often they have both.
- The **rows** attribute specifies the number of rows of frames that will occupy the windows and **cols** attribute specifies the number of columns of frames.
- **There are 3 kinds of possible values for rows: numbers, percentage and asterisk.**
- Normally, two or more values, separated by commas, are given in quoted string.
- **Example:**

i. `<frameset rows="200,300,400">`

Here, if the windows height= 900px, the 3 rows will have heights 200px, 300px and 400px respectively. If not exactly 900px, the actual height will be divided so that the first frame will have 2/9 of the total height, the second 3/9=1/3 of the total height and the last one will have 4/9 of the total height.

ii. `<frameset rows="22%,33%,45%">`

iii. `<frameset rows= "22%,33%, * ">`

Here, asterisk (*) represents the remainder of the windows height.

Representation of Frames:

- The content of frame is specified with `<frame>` tag, which can appear only in the content of frameset element.
- Each frame defined in a frameset has an associated `<frame>` tag that gives the filename of a document that supplies its content.
- The frames in the frameset appear by rows. Example, if frameset has two rows and two columns, the first two frames fill the two columns of the first row of frames.
- The content of frame is specified as the value of the **src** attribute in the `<frame>` tag.
e.g. `<frame src="apples.html">`
- If the `<frame>` tag has no **src** attribute, the browser displays the empty frame.
- If the content of the frame does not fit into the given frame, scroll bars are implicitly included.
- For including scroll bars explicitly, **scrolling** attribute of the `<frame>` tag can be set to **yes**.
- **Note:** If you want to validate a page containing frames, be sure the **<!DOCTYPE>** is set to either "HTML Frameset DTD" or "XHTML Frameset DTD"

Optional Attributes

Attribute	Value	Description
<u>frameborder</u>	0 1	Not supported in HTML5. Specifies whether or not to display a border around a frame
<u>longdesc</u>	<i>URL</i>	Not supported in HTML5. Specifies a page that contains a long description of the content of a frame
<u>marginheight</u>	<i>pixels</i>	Not supported in HTML5. Specifies the top and bottom margins of a frame
<u>marginwidth</u>	<i>pixels</i>	Not supported in HTML5. Specifies the left and right margins of a frame
<u>name</u>	<i>text</i>	Not supported in HTML5. Specifies the name of a frame

<u>noresize</u>	Noresize	Not supported in HTML5. Specifies that a frame is not resizable
<u>scrolling</u>	yes no auto	Not supported in HTML5. Specifies whether or not to display scrollbars in a frame
<u>src</u>	URL	Not supported in HTML5. Specifies the URL of the document to show in a frame

Frame Examples:

1. <!DOCTYPE html>

<html>

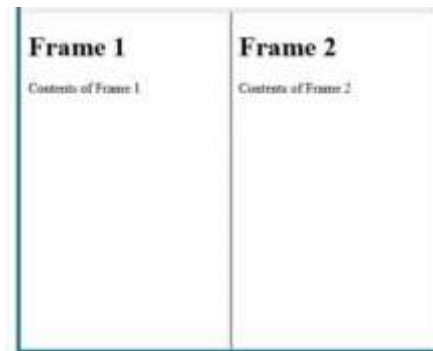
<frameset cols="20%,*,*">

<frame src="frame1.html">

<frame src="frame2.html">

</frameset>

</html>



2.

. <!DOCTYPE html>

<html>

<frameset cols="*,*">

<frame src="frame_1.html">

<frameset rows="*,*">

<frame src="frame_2.html">

<frameset cols="*,*">

<frame src="frame_3.html">

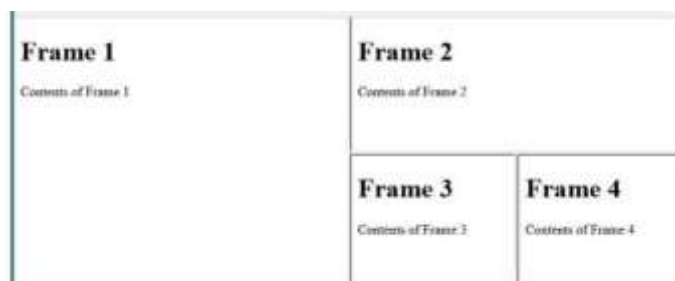
<frame src="frame_4.html">

</frameset>

</frameset>

</frameset>

</html>



Advantages and Disadvantages of Using Frames:

Advantages of Using Frames:

- It allows the user to view multiple documents within a single web page.
- It is possible to load pages from different servers in a single frameset and maintained.
- Different frames can have different properties like- color, background, etc. within the same page.

Disadvantages of Using Frames:

- Frames can make the production of website more complicated, mistakes like- links creating duplicate web pages within the frame may occur.
- Search engines reference a web page within a complete frameset tag, but not the frame pages.
- The use of too many frames can put a high workload on the server.
- Bookmark only works for the top-level pages (the frameset themselves). A user is unable to bookmark any of the web pages viewed within a frame.
- Older browsers do not support frames. Moreover, <frame> tag is not supported in HTML5.

HTML <iframe> Tag

Definition and Usage

The <iframe> tag specifies an inline frame.

An inline frame is used to embed another document within the current HTML document.

Example

An inline frame is marked up as follows:

```
<iframe src="https://www.w3schools.com"></iframe>
```

HTML Multimedia

What is Multimedia?

Multimedia comes in many different formats. It can be almost anything you can hear or see.

Examples: Images, music, sound, videos, records, films, animations, and more.

Web pages often contain multimedia elements of different types and formats.

Browser Support

The first web browsers had support for text only, limited to a single font in a single color.

Later came browsers with support for colors and fonts, and images!

Audio, video, and animation have been handled differently by the major browsers. Different formats have been supported, and some formats require extra helper programs (plug-ins) to work.

Multimedia Formats

Multimedia elements (like audio or video) are stored in media files.

The most common way to discover the type of a file, is to look at the file extension.

Multimedia files have formats and different extensions like: .swf, .wav, .mp3, .mp4, .mpg, .wmv, and .avi.

Format	File	Description
MPEG	.mpg	MPEG. Developed by the Moving Pictures Expert Group. The first popular video format on the web. Used to be supported by all browsers, but it is not supported in HTML5 (See MP4).
	.mpeg	
AVI	.avi	AVI (Audio Video Interleave). Developed by Microsoft. Commonly used in video cameras and TV hardware. Plays well on Windows computers, but not in web browsers.
WMV	.wmv	WMV (Windows Media Video). Developed by Microsoft. Commonly used in video cameras and TV hardware. Plays well on Windows computers, but not in web browsers.
QuickTime	.mov	QuickTime. Developed by Apple. Commonly used in video cameras and TV hardware. Plays well on Apple computers, but not in web browsers. (See MP4)
RealVideo	.rm .ram	RealVideo. Developed by Real Media to allow video streaming with low bandwidths. It is still used for online video and Internet TV, but does not play in web browsers.
Flash	.swf .flv	Flash. Developed by Macromedia. Often requires an extra component (plug-in) to play in web browsers.
Ogg	.ogg	Theora Ogg. Developed by the Xiph.Org Foundation. Supported by HTML5.
WebM	.webm	WebM. Developed by the web giants, Mozilla, Opera, Adobe, and Google. Supported by HTML5.
MPEG-4 or MP4	.mp4	MP4. Developed by the Moving Pictures Expert Group. Based on QuickTime. Commonly used in newer video cameras and TV hardware. Supported by all HTML5 browsers. Recommended by YouTube.

The HTML <video> Element

The HTML5 <video> element specifies a standard way to embed a video in a web page.

To show a video in HTML, use the <video> element:

Example

```
<video width="320" height="240" controls>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">  
Your browser does not support the video tag.  
</video>
```

How it Works

The **controls** attribute adds video controls, like play, pause, and volume.

It is a good idea to always include **width** and **height** attributes. If height and width are not set, the page might flicker while the video loads.

The <source> element allows you to specify alternative video files which the browser may choose from. The browser will use the first recognized format.

The text between the <video> and </video> tags will only be displayed in browsers that do not support the <video> element.

HTML <video> Autoplay

To start a video automatically use the **autoplay** attribute:

Example

```
<video width="320" height="240" autoplay>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">  
Your browser does not support the video tag.  
</video>
```

HTML Video - Media Types

File Format Media Type

MP4	video/mp4
WebM	video/webm
Ogg	video/ogg

HTML Video - Methods, Properties, and Events

HTML5 defines DOM methods, properties, and events for the <video> element.

This allows you to load, play, and pause videos, as well as setting duration and volume.

There are also DOM events that can notify you when a video begins to play, is paused, etc.

Audio Formats

MP3 is the newest format for compressed recorded music. The term MP3 has become synonymous with digital music.

If your website is about recorded music, MP3 is the choice.

Format	File	Description
MIDI	.mid .midi	MIDI (Musical Instrument Digital Interface). Main format for all electronic music devices like synthesizers and PC sound cards. MIDI files do not contain sound, but digital notes that can be played by electronics. Plays well on all computers and music hardware, but not in web browsers.
RealAudio	.rm .ram	RealAudio. Developed by Real Media to allow streaming of audio with low bandwidths. Does not play in web browsers.
WMA	.wma	WMA (Windows Media Audio). Developed by Microsoft. Commonly used in music players. Plays well on Windows computers, but not in web browsers.
AAC	.aac	AAC (Advanced Audio Coding). Developed by Apple as the default format for iTunes. Plays well on Apple computers, but not in web browsers.
WAV	.wav	WAV. Developed by IBM and Microsoft. Plays well on Windows, Macintosh, and Linux operating systems. Supported by HTML5.
Ogg	.ogg	Ogg. Developed by the Xiph.Org Foundation. Supported by HTML5.
MP3	.mp3	MP3 files are actually the sound part of MPEG files. MP3 is the most popular format for music players. Combines good compression (small files) with high quality. Supported by all browsers.
MP4	.mp4	MP4 is a video format, but can also be used for audio. MP4 video is the upcoming video format on the internet. This leads to automatic support for MP4 audio by all browsers.

The HTML <audio> Element

To play an audio file in HTML, use the <audio> element:

Example

```
<audio controls>  
  <source src="horse.ogg" type="audio/ogg">  
  <source src="horse.mp3" type="audio/mpeg">
```

Your browser does not support the audio element.
</audio>

HTML Audio - How It Works

The **controls** attribute adds audio controls, like play, pause, and volume.

The **<source>** element allows you to specify alternative audio files which the browser may choose from. The browser will use the first recognized format.

The text between the <audio> and </audio> tags will only be displayed in browsers that do not support the <audio> element.

HTML Audio - Media Types

File Format	Media Type
MP3	audio/mpeg
Ogg	audio/ogg
Wav	audio/wav

HTML Audio - Methods, Properties, and Events

HTML5 defines DOM methods, properties, and events for the <audio> element.

This allows you to load, play, and pause audios, as well as set duration and volume.

There are also DOM events that can notify you when an audio begins to play, is paused, etc.

HTML Plug-ins

The <object> Element

The <object> element is supported by all browsers.

The <object> element defines an embedded object within an HTML document.

It is used to embed plug-ins (like Java applets, PDF readers, Flash Players) in web pages.

Example

```
<object width="400" height="50" data="bookmark.swf"></object>
```

The <object> element can also be used to include HTML in HTML:

Example

```
<object width="100%" height="500px" data="snippet.html"></object>
```

Or images if you like:

Example

```
<object data="audi.jpeg"></object>
```

The <embed> Element

The <embed> element is supported in all major browsers.

The <embed> element also defines an embedded object within an HTML document.

Web browsers have supported the <embed> element for a long time. However, it has not been a part of the HTML specification before HTML5.

Example

```
<embed width="400" height="50" src="bookmark.swf">
```

HTML versus XHTML

- XHTML (Extensible Hypertext Markup Language) is HTML written as XML.
- HTML has lax (not sufficiently strict) syntax rules, leading to sloppy and sometimes ambiguous documents.
- XHTML syntax is much stricter, leading to clean and clear documents in a standard form.
- Even minor syntax errors will prevent a document labelled as XML from being rendered fully, whereas they would be ignored in the HTML syntax.
- HTML is compatible with most legacy web browsers.

The Most Important Differences of XHTML from HTML:

Document Structure

- XHTML DOCTYPE is **mandatory**
- The xmlns attribute in <html> is **mandatory**
- <html>, <head>, <title>, and <body> are **mandatory**

XHTML Elements

- XHTML elements must be **properly nested**
- XHTML elements must always be **closed**
- XHTML elements must be in **lowercase**
- XHTML documents must have **one root element**

XHTML Attributes

- Attribute names must be in **lower case**
- Attribute values must be **quoted**
- Attribute minimization is **forbidden**

What determines if the document is HTML or XHTML?

The one and only thing that controls whether a document is HTML or XHTML is the MIME type. If the document is served with a **text/html** MIME type, it is treated as HTML. If it is served as **application/xhtml+xml** or **text/xml**, it gets treated as XHTML

UNIT-2

Hyper Text Markup Language

Writing code for given tables & forms is imp in this unit.
OR creating those writing html code.

HTML stands for Hyper Text Markup language. It is the standard markup language for web pages. HTML consists of a series of elements and tells the browser how to display the content. HTML elements are represented by tags. Browsers do not display the HTML tags, but use them to render the content of the page. With the help of HTML we can create the structure of our website. HTML is like skeleton of human body for any website, which gives base to the structure.

⊗ Structure of HTML with example: [Imp]

```
<!DOCTYPE html>
<html>
  <head>
    <title> Page Title </title>
  </head>
  <body>
    <h1> This is a heading. </h1>
    <p> This is a paragraph. </p>
  </body>
</html>
```

This is example & explanation

Output:

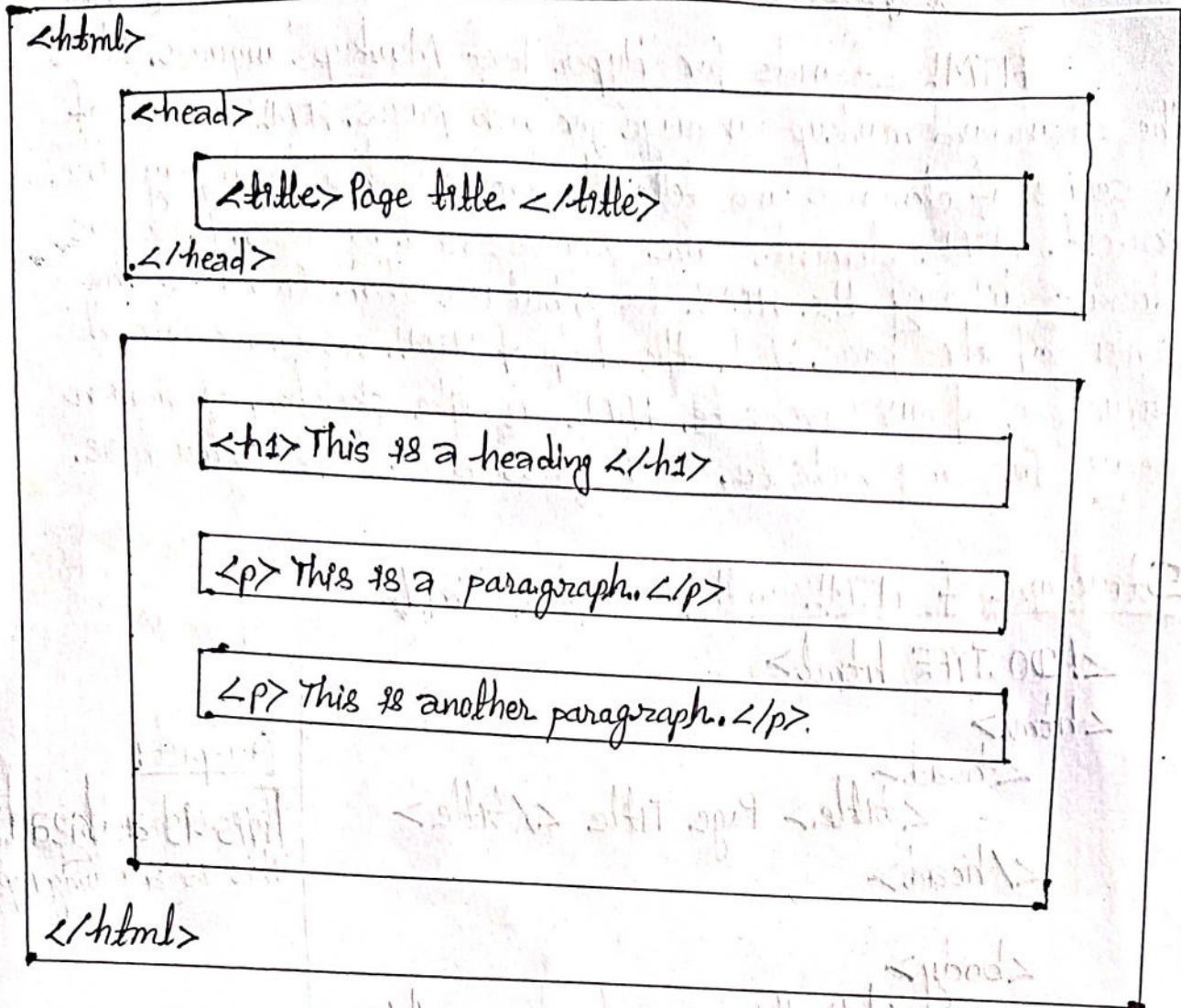
This is a heading.
This is a paragraph.

Note:-

<...> → denotes opening tag.
</...> → denotes closing tag

- The <!DOCTYPE html> declaration defines this document to be HTML5.
- html, head, body, title, h1, p etc. are html elements. The symbol < > denotes opening tag of an element and symbol </> denotes closing tag of an element.
- The <html> element is the root element of an HTML page.
- The <head> element contains <title> element which describes or names the title of document and it also contains meta tags.
- The <body> element contains the visible page content.
- The <h1> element defines a large heading.
- The <p> element defines a paragraph.

HTML Page Structure



⊗. HTML Elements:

HTML element is defined by a start tag, some content, and an end tag. The HTML element is everything from the start tag to the end tag:

<tagname> Content goes here... </tagname>

Examples:

<h1> My First Heading </h1>

<p> My first paragraph. </p>

Here,

Start tag	Element Content	End tag
<h1>	My First Heading	</h1>
<p>	My first paragraph.	</p>
 	none	none

Note: Some HTML elements have no content (like
 element) which provides some space. These are empty elements and do not have end tag.

⊗. HTML Attributes:

All HTML elements can have attributes. Attributes provide additional information about elements. Attributes are always specified in the start tag. Attributes usually come in name/value pairs like: `name="value"`. Following are some of the HTML attributes:

i) The href attribute: The `<a>` tag defines a hyperlink. The `href` attribute specifies the URL of the page the link goes to:

Example: `<a href="https://www.notejunction.com"> Visit NoteJunction </a>`

ii) The src attribute: The `<img>` tag is used to embed an image in an HTML page. The `src` attribute specifies the path to the image to be displayed:

Example: ``

iii) The width and height attributes: The `<img>` tag also contain the `width` and `height` attributes, which specifies the width and height of the image (in pixels).

Example: ``

iv) The alt attribute: The `alt` attribute for the `<img>` tag specifies an alternate text for an image, if the image for some reason cannot be displayed. This can be due to slow connection, or an error in the `src` attribute.

Example: ``

⊗. HTML Headings:

HTML headings are titles or subtitles that we want to display on a webpage. HTML headings are defined with the `<h1>` to `<h6>` tags. `<h1>` defines the most important heading and has the largest font size while `<h6>` defines the least important heading and has the smallest font size. Browsers automatically add some white space (margin) before and after a heading. It is important to use headings to show the document structure. `<h1>` headings should be used for main headings, `<h2>` for sub-headings and so on as per requirement.

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>This is heading 1 </h1>
    <h2>This is heading 2 </h2>
    <h3>This is heading 3 </h3>
    <h4>This is heading 4 </h4>
    <h5>This is heading 5 </h5>
    <h6>This is heading 6 </h6>
  </body>
</html>
```

Output:

This is heading 1
This is heading 2
This is heading 3
This is heading 4
This is heading 5
This is heading 6

⊗. HTML Paragraphs:

The paragraph always starts on a new line, and is usually a block of text. The html `<p>` element defines a paragraph. It also adds some white space (margin) before and after a paragraph.

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <p>This is a paragraph. </p>
    <p>This is another paragraph. </p>
  </body>
</html>
```

Output:

This is a paragraph.
This is another paragraph.

⊗. HTML Division:

The `<div>` tag defines a division or a section in an HTML document. The `<div>` tag is used as a container for HTML elements which is then styled with CSS or manipulated with JavaScript. The `<div>` tag is easily styled by using the class or id attribute. Any sort of content can be put inside the `<div>` tag like html headings, html paragraphs, html forms, html tables etc. By default, browsers always place a line break before and after the `<div>` element.

⊗ HTML Comments:

HTML comments are not displayed in the browser, but they can help document our HTML source code. We can add comments to our HTML source by using following syntax:

`<!-- Write comments here -->`

Comments can be used to hide content which may help us to write things useful to understand code in better way but it should be hidden from document or it should not be executed. We can hide one line or more than one line, everything between the `<!--` and the `-->`.

⊗ Formatting:

HTML formatting is used for formatting text. Formatting elements were designed to display special types of text.

`<b>` → Bold text

`<strong>` → Important text

`<i>` → Italic text

`<em>` → Emphasized text

`<mark>` → Marked text

`<small>` → Smaller text

`<del>` → Deleted text

`<ins>` → Inserted text

`<sub>` → Subscript text

`<sup>` → Superscript text.

1) HTML `<b>` and `<strong>` elements:

The HTML `<b>` element defines bold text, without any extra importance.

Example: `<b>`This text is bold.`</b>`

The HTML `<strong>` element defines text with strong importance.

The content inside is typically displayed in bold.

Example: `<strong>`This text is important!`</strong>`

2) HTML `<i>` and `<em>` elements:

The HTML `<i>` element defines a part of text in an alternate mood. The content inside is typically displayed in italic.

Example: `<i>`This text is italic.`</i>`

The HTML `<em>` element defines emphasized text. The content inside is typically displayed in italic. Example: `<em>`This text is emphasized.`</em>`

iii) HTML <small> element:

The HTML **<small>** element defines smaller text.

Example: **<small>**This is some smaller text.**</small>**

iv) HTML <sub> element:

The HTML **<sub>** element defines subscripted text. Subscript text appears half a character below the normal line and is sometimes rendered in a smaller font. Subscript text can be used for displaying chemical formulas, like H_2O .

Example: **<p>**H **_{**2**}** O **</p>**

v) HTML <sup> element:

The HTML **<sup>** element defines superscript text. Superscript text appears half a character above the normal line, and is sometimes rendered in a smaller font. Superscript text can be used for footnotes, like WWW¹.

Example: **<p>**WWW **^{**1**}** **</p>**

Spacing:

Spacing is used to maintain space between texts. We have several options for creating and controlling white space on our website following are some of them;

i) HTML
 element:

The HTML **
** tag denotes a line break. We use this tag if we want to display upcoming text in the next line.

ii) HTML <pre> element:

The HTML **<pre>** tag is used with preformatted text. It instructs the browser that the text is to appear exactly as written in the HTML file, including any spaces or blank lines. If we write five spaces inside **<pre>** tags, then we get five spaces on the website.

iii) ** ** → This will add single white space.

iv) ** ** → This will add two white spaces.

v) ** ** → This will add four white spaces.

Formatting text phrases

- i) The element: The `<span>` tag is an inline container used to mark up a part of a text, or a part of a document. The `<span>` tag is much like `<div>` element, but `<div>` is a block-level element and `<span>` is an inline element. The `<span>` tag supports global and event attributes.
- ii) The <tt> element: The `<tt>` tag was used in HTML 4 to define teletype text.

⊗. Image Element:

The HTML `<img>` tag is used to embed an image in a web page. Images are not technically inserted into a web page rather images are linked to web pages. The `<img>` tag creates a holding space for the referenced image. The `<img>` tag is empty, it contains attributes only, and does not have a closing tag. The `<img>` tag has two required attributes:

src → specifies the path to the image
alt → specifies the alternate text for image.

Syntax:

```

```

Example:

```

```

When a web page loads, the browser gets the image from server or local file and inserts into the page. The src attribute should be given correct path or url, otherwise we will get a broken link icon on the browser, along with alt text value.

If the user for some reason cannot view image because of slow connection or an error in the src attribute or any other reason then the value of alt attribute describes the image. We can use `style` attribute to specify width and height of image.

Example: ``

Images in another folder:

If we have images in a sub-folder, we must include folder name in the **src** attribute as in the example below:

Example: ``

Images on another server/website:

Some websites point or display an image that is on another server/website. To use or point an image on another server, we must specify full url in the **src** attribute:

Example: ``

Background Image:

A background image can be specified for almost any HTML element. To add a background image on an HTML element, use the HTML **style** attribute and the CSS **background-image** property:

Example: `<div style="background-image: url('img-roshan.jpg');">`

⊗ Anchor tag (<a> tag):

An Anchor tag in HTML can be defined as a means to create a hyperlink that can link our current page on which the text is being converted to hypertext via `<a>` (anchor tag) to another page. This anchoring from one page to another is made possible by the attribute "href".

The attribute "href" of the anchor tag is implemented for defining the address or path to which this hypertext will get linked. The syntax for an anchor tag with href attribute looks something like this:

`<a href="URL">Text Here </a>`

- An unvisited link gets displayed with properties like underlined and with the color blue.
- A link that is visited gets displayed as underlined with color as purple.
- A link that is an active link gets displayed as underlined with red color.

⊗. Lists:

HTML lists allow web developers to group a set of related items in lists.

1) Unordered list: An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag. The list items will be marked with bullets (small black circles) by default:

Example:

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Output:

- Coffee
- Tea
- Milk

2) Ordered list: An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag. The list items will be marked with numbers by default:

Example:

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Output:

1. Coffee
2. Tea
3. Milk

Note:

1) Unordered list items are marked with circles by default but we can change it specifying `style` attribute and `list-style-type` property with values like circle, square, none etc. in the `<ul>` tag.

Example: `<ul style="list-style-type: square">`

```
<li>Tea</li>
<li>Milk</li>
</ul>
```

Output:

- Tea
- Milk

2) Similarly ordered list are marked with numbers but we can change it specifying `type` attribute in `<ol>` tag.

Example: `<ol type="i">`

```
<li>Tea</li>
<li>Milk</li>
</ol>
```

Output:

- i. Tea
- ii. Milk

⊗. Tables: [Imp]

HTML table allows web developers to arrange data into rows and columns. It is defined with the `<table>` tag. Each A table header is defined with the `<th>` tag. Each table row is defined with the `<tr>` tag. By default, table headings are bold and centered. A table data/cell is defined with the `<td>` tag.

Example:

```
<table>
  <tr>
    <th>Company </th>
    <th>Contact </th>
    <th>Country </th>
  </tr>
  <tr>
    <td>A1 Soft </td>
    <td>Sandeep Singh </td>
    <td>Nepal </td>
  </tr>
  <tr>
    <td>Island Trading </td>
    <td>Helen Bennet </td>
    <td>UK </td>
  </tr>
</table>
```

Output:

Company	Contact	Country
A1 Soft	Sandeep Singh	Nepal
Island Trading	Helen Bennet	UK

Company Contact Country
A1 Soft Sandeep Singh Nepal
Island Trading Helen Bennet UK

A border is set using the CSS border property. If we do not specify a border for the table, it will be displayed without borders. Let us use CSS property in the above example as below:

```
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
```

Now the Output with CSS will be as follows:

Company	Contact	Country
A1 Soft	Sandeep Singh	Nepal
Island Trading	Helen Bennet	UK

Cell Padding → `cellpadding` attribute can be specified in the `<table>` to get space between the cell content and its borders.

Example:- `<table cellpadding="5px">`

`</table>`

Align → Table headings and table data can be align aligned to left, right, center etc using css property `text-align`. Let we have to align table headings to center then it can be done as;

```
th {
  text-align: center;
}
```

Column Span (colspan): To make a cell span more than one column, we use the `colspan` attribute.

Example: `<table style="width:100%">`

`<tr>`

`<th>Name </th>`

`<th colspan="2">`

Telephone

`</th>`

`</tr>`

`<tr>`

`<td>Bill Gates </td>`

`<td>55578546 </td>`

`<td>57584567 </td>`

`</tr>`

`</table>`

Output:

Name	Telephone	
Bill Gates	55578546	57584567

Row span (rowspan): To make a cell span one or more than one row, we use ~~rowspan~~ **rowspan** attribute.

Example:

<table style="width:100%">

<tr>

<th>Name </th>

<td>Bill Gates </td>

<tr>

<tr>

<th rowspan="2">

Telephone

</th>

<td>55577854 </td>

</tr>

<tr>

<td>55577855 </td>

</tr>

</table>

looks similar to colspan but not th td and tr are arranged in a way so that it looks like proper table

Output:

Name	Bill Gates
Telephone	5577854
	5577855

Table Caption: We can add caption that serves as a heading for the entire table. To add caption to a table, we use the **<caption>** tag immediately after the **<table>** tag.

Example:

<table style="width:100%">

<caption>Monthly Savings </caption>

<tr>

<th>Month </th>

<th>Savings </th>

<tr>

<td>January </td>

<td>₹100 </td>

</tr>

<tr>

<td>February </td>

<td>₹50 </td>

</tr>

</table>

Output:

Monthly Savings	
Month	Savings
January	₹100
February	₹50

11

⊗. HTML Frames (iframe): The "iframe" in html stands for inline frame. The "iframe" tag defines a rectangular region within the document in which the browser can display a separate document, including scrollbars and borders. The "src" attribute is used to specify the URL of the document that occupies the frame.

Syntax: `<iframe src="URL" title="description"> </iframe>`

Many times we see youtube videos are embedded within news portal and other websites. This is done with the help of iframe. The embedded link of youtube video is place in the src of iframe to make this all happen.

⊗. Forms: [Imp]

HTML form is used to collect user input. The user input is most often sent to a server for processing. The HTML `<form>` element is used to create an HTML form for user input. The `<form>` element is a container for different types of input elements such as; text, checkboxes, radio, submit buttons etc.

1) The <input> Element: The HTML `<input>` element is most used form element. An `<input>` element can be displayed in many ways, depending on type attribute. Here are some examples:

<u>Type</u>	<u>Description</u>
<code><input type="text"></code>	Displays a single-line text input field.
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices).
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices).
<code><input type="submit"></code>	Displays a submit button.
<code><input type="button"></code>	Displays a clickable button.

2) The <label> Element: The `<label>` tag defines a label for many form elements. The `<label>` element is useful for screen-reader users. to distinguish between different fields like username, password etc.

Example: A form with radio buttons:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Radio Buttons </h2>
```

```
<p>Choose your favourite web language: </p>
```

```
<form>
```

```
<input type="radio" id="html" name="fav_language" value="HTML">
```

```
<label for="html"> HTML </label> <br>
```

```
<input type="radio" id="css" name="fav_language" value="CSS">
```

```
<label for="css"> CSS </label> <br>
```

```
<input type="radio" id="javascript" name="fav_language" value="JavaScript">
```

```
<label for="javascript"> JavaScript </label>
```

```
</form>
```

```
</body>
```

```
</html>
```

Output:

Radio Buttons

Choose your favourite web language:

☐ HTML

☐ CSS

☐ JavaScript.

⊗. HTML <form> Elements:

The HTML <form> element can contain one or more of the following form elements:

<input>

<label>

<select>

<textarea>

<button>

<fieldset>

<legend>

<datalist>

<output>

<option>

<optgroup>

3) The `<select>` Element:

The `<select>` element defines a drop-down list.

Example:

```
<form>
  <label for="cars"> Choose a car: </label>
  <select>
    <option value="volvo"> Volvo </option>
    <option value="saab"> Saab </option>
    <option value="fiat"> Fiat </option>
    <option value="audi"> Audi </option>
  </select>
  <input type="submit">
</form>
```

4) The `<option>` Element:

The `<option>` elements defines an option that can be selected. By default, the first item in the drop-down list is selected. To define a pre-selected option, add the ~~selected~~ `selected` attribute to the option.

Example: `<option value="audi" selected> Fiat </option>`

We use the `size` attribute to specify the number of visible values.

```
Example: <select size="3">
  <option> ... </option>
  <option> ... </option>
  ...
  <option> ... </option>
</select>
```

Similarly we use `multiple` attribute to allow the user to select more than one value.

5) The `<textarea>` Element:

The `<textarea>` element defines a multi-line input field. The `rows` attribute specifies the visible number of lines in a text area. The `cols` attribute specifies the visible width of a text area.

Example: `<textarea name="message" rows="10" cols="30">`
 The cat was playing in the garden.
`</textarea>`

⊗. Form Attributes:

i) The **action** attribute: The **<action>** attribute defines the action to be performed when the form is submitted. Usually, the form data is sent to a file on the server when the user clicks on the submit button. In the example below, the form data is sent to a file called "action_page.php". This file contains a server-side script that handles the form data:

Example: **<form action="/action_page.php">**
: **<!-- form elements -->**
</form>

If the **<action>** attribute is omitted, the action is set to the current page.

ii) The **target** attribute: The **target** attribute specifies where to display the response that is received after submitting the form. The **target** attribute has default value **_self** which means that the response will open in the current window. The **target** attribute can have one of the following values:

<u>Value</u>	<u>Description</u>
_blank	The response is displayed in new tab.
_self	The response is displayed in current tab.
_parent	The response is displayed in parent frame.
_top	The response is displayed in full body of window or tab.

Example: Here, the submitted result will open in a new browser tab:

<form action="/action_page.php" target="_blank">

iii) The **method** attribute: The **method** attribute specifies the HTTP method to be used when submitting the form data. The form-data can be sent as URL variables (with **method="get"**) or as HTTP post transaction (with **method="post"**). The default HTTP method when submitting form data is GET.

Example: **<form action="/action_page.php" method="get">**

94) The autocomplete Attribute:

The **autocomplete** attribute specifies whether a form should have autocomplete on or off. When autocomplete is on, the browser automatically complete values based on values that the user has entered before.

Example: `<form action="/action_page.php" autocomplete="on">`

95) The novalidate Attribute:

The **novalidate** attribute is a boolean attribute. When present, it specifies that the form-data (input) should not be validated when submitted.

Example: `<form action="/action_page.php" novalidate>`

96) HTML Input Types:

`<input type="button">`
`<input type="checkbox">`
`<input type="color">`
`<input type="date">`
`<input type="datetime-local">`
`<input type="email">`
`<input type="file">`
`<input type="image">`
`<input type="number">`

`<input type="password">`
`<input type="radio">`
`<input type="range">`
`<input type="reset">`
`<input type="search">`
`<input type="submit">`
`<input type="tel">`
`<input type="text">`
`<input type="url">`
`<input type="week">`

97) HTML Input value Attributes:

97) **value** attribute:- The input **value** attribute specifies an initial value for an input field.

Example: `<input type="text" value="John">`

98) **readonly** attribute:- The input **readonly** attribute specifies that an input field is read-only it can't be modified.

Example: `<input type="text" value="John" readonly>`

99) **disabled** attribute:- The input **disabled** attribute specifies that an input field should be disabled. A disabled field is unusable and un-clickable. The value of a disabled input field will not be sent when submitting the form.

Example: `<input type="text" value="John" disabled>`

v) size attribute: The input **size** attribute specifies the visible width, in characters, of an input field. The default value for **size** is 20.

Example: `<input type="text" name="pm" size="4">`

v) The min and max attributes: The input **min** and **max** attributes specify the minimum and maximum values for an input field.

Example: `<input type="number" min="1" max="5">`

v) placeholder attribute: The input **placeholder** attribute specifies a short hint that describes the expected value of an input field.

v) required attribute: The input **required** attribute specifies that an input field must be filled out before submitting the form.

 Based on form knowledge try to practice login and signup forms. In exam theory is not asked, in many cases creating login and signup forms or other forms are asked. Go through youtube tutorials it's easy. [theory is for base knowledge only]!

⊗ Meta Tag:

The **<meta>** tag defines metadata about an HTML document. Metadata is information about data. **<meta>** tags always go inside the **<head>** element, and are typically used to specify character set, page description, keywords, author of the document, and viewport settings. Metadata will not be displayed on the ~~paged~~ page, but is machine parsable. Metadata is used by browsers, search engines and other web services.

Examples: i) Define keywords for search engines:

`<meta name="keywords" content="HTML, CSS">`

ii) Refresh document in every 30 seconds.

`<meta http-equiv="refresh" content="30">`

iii) Setting the viewport to make website look good on all devices:

`<meta name="viewport" content="width=device-width, initial-scale=1.0">`

⊗ HTML Audio:

The HTML `<audio>` element is used to play an audio file on web page.

Example:-

`<audio controls>`

`<source src="horse.ogg" type="audio/ogg">`

`<source src="horse.mp3" type="audio/mpeg">`

`</audio>`

The `controls` attribute adds audio controls like play, pause and volume. The `<source>` element allows us to specify path or url to audio we are going to use. The browser will use the first recognized format. To start an audio file automatically we use `autoplay` attribute. Similarly we can use `muted` attribute to mute the audio. The `preload` attribute specifies how the audio should be loaded when the page loads.

Preload Syntax: `<audio preload="auto/metadata/none">`

where, `auto` → Entire audio file is loaded when page loads.

`metadata` → The browser should load only metadata when page loads.

`none` → The browser should not load audio file when page loads.

The `loop` attribute is a boolean attribute. When present, it specifies that the audio will start over again, every time it is finished.

⊗ HTML Video:

The HTML `<video>` element is used to show a video on web page.

Example:

`<video width="320" height="240" controls>`

`<source src="movie.mp4" type="video/mp4">`

`</video>`

The `controls` attribute adds video controls like play, pause and volume. It is good idea to always include `width` and `height` attributes. If height and width are not set, the page might flicker while the video loads. The `<source>` element allows us to specify path or url to video we are going to use. To start video automatically, we use `autoplay` attribute. The `loop` attribute is a boolean attribute. When present, it specifies that the video will start over again, every time it is finished. The `poster` attribute is used to display the image while video downloading or when user click the play button. If this image is not set then it will take first frame of video as poster image. Syntax: `<video poster="URL">`

⊗. Canvas:

The HTML `<canvas>` element is used to draw graphics on a web page. The `<canvas>` element is only a container for graphics. We must use Javascript to actually draw the graphics. Canvas has several methods for drawing paths, boxes, circles, text and adding images. A canvas is a rectangular area on an HTML page. By default, a canvas has no border and no content.

Example: Drawing a line in canvas:

```
<canvas id="myCanvas" width="200" height="100"></canvas>
```

Adding JavaScript

```
<script>
```

```
var c=document.getElementById("myCanvas");
```

```
var ctx=c.getContext("2d");
```

```
ctx.moveTo(0,0);
```

```
ctx.lineTo(200,100);
```

```
ctx.stroke();
```

```
</script>
```

for circle we will change these two lines as:

```
ctx.beginPath();
```

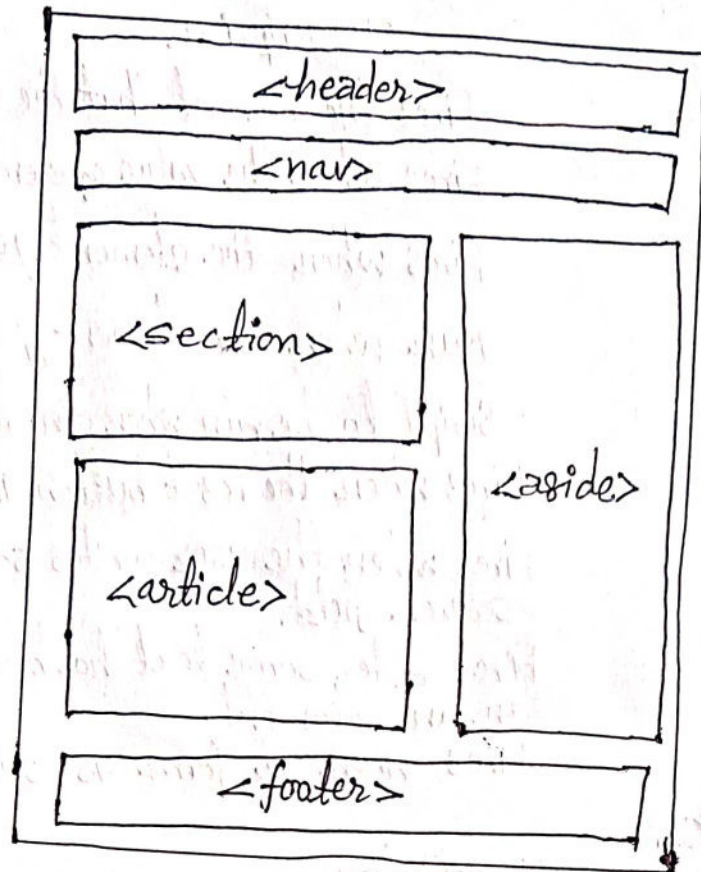
```
ctx.arc(50,50,40,0,2*Math.PI);
```

others all same

⊗. HTML Semantic Elements:

A semantic element clearly describes its meaning to both the browser and the developer. `<div>` and `<span>` are examples of non-semantic elements which tells nothing about its content. `<form>`, `<table>` and `<article>` are some examples of semantic elements which clearly defines its content. Following are some semantic elements used in HTML:

- i) `<article>` → defines independent, self-contained content.
- ii) `<aside>` → defines content aside from page content.
- iii) `<details>` → defines additional details that user can view or hide.
- iv) `<footer>` → defines a footer for a document.
- v) `<header>` → defines a header for a document.
- vi) `<main>` → specifies the main content of a document.
- vii) `<nav>` → defines navigation links.
- viii) `<section>` → defines a section in document.



Fires meaning runs
✓

⊗ HTML Events:

1) Window Events: HTML has the ability to let events trigger actions in a browser, like starting a Javascript when a user clicks on an element. Events triggered for the window object applies to the **<body>** tag;

Attribute

Description

onerror

Script to be run when error occurs

onload

Script to be ~~load~~ run after page finishes loading.

onmessage

Script to be run when message is triggered.

onoffline

Script to be run when browser starts to work offline.

ononline

Script to be run when browser starts to work online.

onresize

Fires when the browser window is resized.

onafterprint

Script to be run after document is printed.

onbeforeprint

Script to be run before document is printed.

2) Form Events: Events triggered by actions inside a HTML form (applies to almost all HTML elements, but is most used in form elements):

Attribute

onblur

onchange

onfocus

oninput

oninvalid

onreset

onsearch

onselect

onsubmit

Description

Fires the moment that the element loses focus.

Fires when the value of element is changed.

Fires when the element gets focus.

Run when an element gets user input

Script to be run when an element is invalid.

Fires when the reset button in form is clicked.

Fires when the user writes something on search field.

Fires after some text has been selected in an element.

Fires when a form is submitted.

3) Keyboard Events:

Attribute

onkeydown

onkeypress

onkeyup

Description

Fires/runs when a user pressing a key.

Fires when user presses key.

Fires when user releases a key.

4) Mouse Events:

Attribute

onclick

ondblclick

onmousedown

onmouseout

onmouseover

onmouseup

~~onmousewheel~~

onmousemove

onwheel

Description

Fires on a mouse click on the element.

Fires on a mouse double-click on the element.

Fires when a mouse is pressed down on an element.

Fires when mouse pointer moves out of an element.

Fires when mouse pointer moves over an element.

Fires when mouse button is released over an element.

Fires when the mouse pointer is moving while it is over an element.

Fires when the mouse wheel rolls up or down over an element.

HTML <marquee> Tag

- The <marquee> tag is used to create scrolling text, images, or other content on a webpage.
- It allows content to move horizontally or vertically across the screen.
- It was commonly used in early web development for announcements, advertisements, and news headlines.
- The movement occurs automatically without using JavaScript.
- The tag is now deprecated in HTML5.

Syntax

```
<marquee>  
    Scrolling Content  
</marquee>
```

Attributes of <marquee> tag

Attribute	Description	Possible Values / Example
direction	Specifies the direction of movement	left, right, up, down
behavior	Defines scrolling behavior	scroll, slide, alternate
scrollamount	Controls scrolling speed	scrollamount="10"
scrolldelay	Sets delay between movements	scrolldelay="100"
loop	Specifies number of repetitions	loop="3"
bgcolor	Sets background color	bgcolor="yellow"
width	Defines marquee width	width="500"
height	Defines marquee height	height="100"
hspace	Sets horizontal space around marquee	hspace="20"
vspace	Sets vertical space around marquee	vspace="20"
align	Aligns marquee relative to surrounding content	top, middle, bottom

Example

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Marquee Tag Example</title>  
</head>  
<body>  
<marquee direction="right" behavior="alternate" scrollamount="8" scrolldelay="100"  
    loop="5" bgcolor="lightblue" width="700"  
    height="100" hspace="20" vspace="20">  
    Welcome to Web Technology Class  
</marquee>  
</body>  
</html>
```

Advantages of <marquee> Tag

1. Simple and easy to use
2. Creates animation without JavaScript
3. Makes webpages more attractive
4. Supports scrolling text and images
5. Suitable for beginners
6. Requires very little code
7. Allows different movement directions

Disadvantages of <marquee> Tag

1. Deprecated in HTML5
2. Not recommended for modern websites
3. Limited customization features
4. Browser behavior may vary
5. Not responsive for modern devices
6. Creates poor user experience in some cases

Applications of <marquee> Tag

1. Displaying breaking news
2. Advertisement banners
3. Website announcements
4. Sports score updates
5. Scrolling image galleries
6. Event notifications
7. Welcome messages on webpages

Why <marquee> is Deprecated

1. It is not part of modern HTML standards.
2. It creates accessibility problems for users.
3. Moving content can distract readers.
4. CSS and JavaScript provide better animation techniques.
5. It lacks flexibility and responsiveness.
6. Different browsers may not support it consistently.
7. Modern animation methods are more professional and customizable.

Modern Alternative

1. CSS Animation
2. JavaScript Animation
3. Bootstrap Carousel
4. Image Slider Libraries

Frames in HTML

In HTML, **frames** refer to separate, independent sections of a webpage that can display individual HTML documents. Frames are often used to divide a webpage into multiple sections, each capable of displaying distinct content, allowing for dynamic, multi-pane layouts.

Historically, frames were popular in web design because they enabled certain portions of a webpage to remain static (e.g., navigation menus or headers) while other parts could change without reloading the entire page. This approach created a more fluid and efficient user experience in some contexts.

However, with the evolution of web technologies and the introduction of **HTML5**, frames have been **deprecated** and replaced by more modern techniques such as **CSS Grid** and **CSS Flexbox**. Despite this, frames are still relevant for understanding older web development practices.

A **frame-based webpage** in HTML is composed of two main elements: **<frameset>** and **<frame>**. The **<frameset>** tag replaces the **<body>** tag in a frame-based document and defines how the browser window is divided into multiple sections, while the **<frame>** tag defines individual frames that display specific HTML documents.

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Target Frames</title>
</head>
<frameset cols="200, *">
  <frame src="menu.html" name="navFrame" />
  <frame src="content.html" name="contentFrame" />
</frameset>
<body>
  Your browser does not support frames.
</body>
</frameset>
</html>
```

Attribute of frameset tag

Attribute	Description	Example
cols	Specifies the number of vertical divisions (columns) within the frameset. It defines the width distribution of the frames. Values can be expressed as pixel values, percentages, or a combination of both.	<code><frameset cols="25%, 75%"></code>
rows	Specifies the number of horizontal divisions (rows) within the frameset. It defines the height distribution of the frames. Like cols, values can be in pixels, percentages, or a combination.	<code><frameset rows="50%, 50%"></code>
frameborder	Determines the presence of a border around the individual frames. A value of 1 (or yes) enables borders, while 0 (or no) disables them. This is a boolean attribute.	<code><frameset cols="50%, 50%" frameborder="1"></code>
border	Specifies the width of the border between frames. The value is expressed in pixels, allowing for control over the frame's visual separation.	<code><frameset cols="50%, 50%" border="5"></code>
frameSpacing	Defines the amount of space (in pixels) between adjacent frames within the frameset. This attribute provides a visual separation between frames.	<code><frameset cols="50%, 50%" frameSpacing="10"></code>
noresize	Prevents user interaction to resize the frames by dragging their borders. When present, this attribute locks the dimensions of the frames, ensuring a fixed layout.	<code><frameset rows="50%, 50%" noresize></code>
name	Assigns a unique identifier to the frameset. This attribute allows reference to the frameset from JavaScript or other elements within the document, although its usage is uncommon in modern web development.	<code><frameset name="mainFrameset"></code>
onresize	A JavaScript event handler that is triggered when the frameset is resized. This allows the developer to execute custom JavaScript code in response to user-initiated resizing actions.	<code><frameset rows="50%, 50%" onresize="alert('Frameset resized!')"></code>
onload	A JavaScript event handler that is executed when the frameset has completed loading. It can be used to run scripts or perform actions once the frameset and its content are fully loaded.	<code><frameset cols="50%, 50%" onload="alert('Frameset loaded!')"></code>

scrolling	Specifies the behavior of scrollbars within the frames. The values yes, no, and auto control whether scrollbars are always visible, never visible, or only appear when necessary, respectively.	<frameset cols="50%, 50%" scrolling="no">
-----------	---	--

HTML Form Elements and Their Types

HTML forms are an essential part of web development, allowing users to input and submit data to a server for processing. Understanding the different form elements and their types is crucial for creating functional and user-friendly web forms. Below is an overview of common HTML form elements and their types.

<input> Element

The `<input>` element is one of the most versatile form elements. It defines various types of input fields that allow users to enter different types of data. The **type** attribute determines the kind of input field.

type="text": Creates a single-line text input field for user data (e.g., name, email).

Example:

```
<input type="text" name="username" placeholder="Enter your username">
```

type="password": Hides the text entered, useful for sensitive information such as passwords.

Example:

```
<input type="password" name="password" placeholder="Enter your password">
```

type="email": Used for email input, with built-in validation for correct email format.

Example:

```
<input type="email" name="email" placeholder="Enter your email">
```

type="number": Restricts input to numeric values, useful for entering quantities or ages.

Example:

```
<input type="number" name="quantity" min="1" max="10">
```

type="checkbox": Creates a checkbox, allowing users to select one or more options.

Example:

```
<input type="checkbox" name="subscribe" value="yes"> Subscribe to newsletter
```

type="radio": Defines radio buttons where the user can select only one option from a group.

Example:

```
<input type="radio" name="gender" value="male"> Male  
<input type="radio" name="gender" value="female"> Female
```

type="submit": Creates a button that submits the form data to the server.

Example:

```
<input type="submit" value="Submit">
```

type="reset": Resets the form to its initial state, clearing all entered data.

Example:

```
<input type="reset" value="Reset">
```

type="file": Allows users to select a file from their device for upload.

Example:

```
<input type="file" name="fileUpload">
```

<textarea> Element

The <textarea> element is used for multi-line text input. Unlike the <input> element, which is for single-line text, <textarea> allows users to enter larger blocks of text, such as comments or descriptions.

Example:

```
<textarea name="feedback" rows="5" cols="40" placeholder="Enter your  
feedback"></textarea>
```

<select> Element

The <select> element creates a dropdown list from which users can select one or more options. Each option within the list is defined using the <option> tag.

Example:

```
<select name="country">  
  <option value="usa">United States</option>  
  <option value="canada">Canada</option>
```

```
<option value="uk">United Kingdom</option>
</select>
```

<optgroup> Element: Used to group related options within a dropdown for better organization.

Example:

```
<select name="country">
  <optgroup label="North America">
    <option value="usa">United States</option>
    <option value="canada">Canada</option>
  </optgroup>
  <optgroup label="Europe">
    <option value="uk">United Kingdom</option>
    <option value="france">France</option>
  </optgroup>
</select>
```

<button> Element

The <button> element defines a clickable button. It can be used for submitting forms or triggering custom JavaScript actions.

Example:

```
<button type="submit">Submit Form</button>
```

<label> Element

The <label> element is used to define a label for a form control, such as a text input or checkbox. It improves accessibility by associating text with a specific input element.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

<fieldset> Element

The <fieldset> element groups related form elements together, making forms easier to organize and understand. It is often used with the <legend> element to provide a title or description for the group of elements.

Example:

```
<fieldset>
  <legend>Personal Information</legend>
  <label for="firstName">First Name:</label>
  <input type="text" name="firstName"><br>
  <label for="lastName">Last Name:</label>
  <input type="text" name="lastName">
</fieldset>
```

<legend> Element

The <legend> element is used to define a title or description for a group of related form elements inside a <fieldset>. It helps improve form readability and accessibility.

Example:

```
<fieldset>
  <legend>Account Information</legend>
  <!-- Form elements here -->
</fieldset>
```

<datalist> Element

The <datalist> element provides a list of predefined options for an <input> element. This feature enables the browser to suggest options based on user input.

Example:

```
<input list="browsers" name="browser" id="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

<input type="range"> Element

The <input type="range"> element allows users to select a value from a specified range by sliding a control. It is useful for selecting values such as volume or ratings.

Example:

```
<input type="range" name="volume" min="0" max="100" value="50">
```

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Comprehensive HTML5 Form Example</title>
</head>
<body>
<h2>Comprehensive HTML5 Form</h2>
<form action="/submit-form" method="POST">

<!-- Text Input -->
<label for="fullname">Full Name:</label>
<input type="text" id="fullname" name="fullname" required placeholder="Enter your full
name" minlength="3" maxlength="50" pattern="[A-Za-z\s]+" title="Name should only
contain letters and spaces"><br><br>

<!-- Email Input -->
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" required placeholder="Enter your email
address"><br><br>

<!-- Password Input -->
<label for="password">Password:</label>
<input type="password" id="password" name="password" required minlength="8"
title="Password must be at least 8 characters long"><br><br>

<!-- Number Input -->
<label for="age">Age:</label>
<input type="number" id="age" name="age" required min="18" max="100" step="1"
placeholder="Enter your age"><br><br>

<!-- Date Input -->
<label for="dob">Date of Birth:</label>
<input type="date" id="dob" name="dob" required><br><br>
```

```
<!-- Time Input -->
<label for="appointment-time">Appointment Time:</label>
<input type="time" id="appointment-time" name="appointment-time" required><br><br>

<!-- Color Input -->
<label for="color">Favorite Color:</label>
<input type="color" id="color" name="color" value="#ff0000"><br><br>

<!-- URL Input -->
<label for="website">Website URL:</label>
<input type="url" id="website" name="website"
placeholder="http://www.example.com"><br><br>

<!-- Phone Number Input -->
<label for="phone">Phone Number:</label>
<input type="tel" id="phone" name="phone" required pattern="\d{10}$" title="Phone
number must be 10 digits"><br><br>

<!-- Checkbox Input -->
<label for="terms">I agree to the terms and conditions:</label>
<input type="checkbox" id="terms" name="terms" required><br><br>

<!-- Radio Button -->
<label for="gender">Gender:</label><br>
<input type="radio" id="male" name="gender" value="male" required>
<label for="male">Male</label><br>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label><br><br>

<!-- Select Dropdown -->
<label for="country">Country:</label>
<select id="country" name="country" required>
    <option value="">Select your country</option>
    <option value="USA">USA</option>
    <option value="Canada">Canada</option>
    <option value="India">India</option>
</select><br><br>

<!-- Textarea -->
<label for="message">Message:</label>
<textarea id="message" name="message" rows="4" cols="50" placeholder="Write your
message here" required></textarea><br><br>
```

```
<!-- File Upload -->
<label for="file-upload">Upload File:</label>
<input type="file" id="file-upload" name="file-upload" accept=".jpg,.png,.pdf"
multiple><br><br>

<!-- Hidden Input (used for tracking purposes or data not meant to be seen by the user) -->
<input type="hidden" name="userID" value="12345"><br><br>

<!-- Submit Button -->
<input type="submit" value="Submit Form">

</form>
</body>
</html>
```

HTML5 Elements and Attributes Used

<input type="text">: Collects textual input (Full Name).

Attributes: required, minlength, maxlength, pattern, title (validation for text input).

<input type="email">: Collects an email address with built-in validation for the email format.

Attribute: required (ensures the field cannot be empty).

<input type="password">: Collects a password with validation for minimum length.

Attribute: required, minlength, title (password must be at least 8 characters).

<input type="number">: Collects numeric input (age).

Attributes: required, min, max, step (age must be between 18 and 100).

<input type="date">: Collects a date input (Date of Birth).

Attribute: required (ensures the field is filled out).

<input type="time">: Collects time input (appointment time).

Attribute: required (ensures the field is filled out).

<input type="color">: Collects a color value from the user (favorite color).

Attribute: value="#ff0000" (sets a default color).

<input type="url">: Collects a website URL.

Attribute: placeholder (sets a hint for the user).

<input type="tel">: Collects a phone number with a regular expression pattern to ensure it's a valid 10-digit phone number.

Attribute: pattern, title (phone number must match the pattern of exactly 10 digits).

<input type="checkbox">: Collects user agreement to terms and conditions.

Attribute: required (ensures checkbox is checked).

<input type="radio">: Collects gender information (select one option).

Attribute: required (ensures one radio button is selected).

<select>: Collects a country selection from a dropdown.

Attribute: required (ensures an option is selected).

<textarea>: Collects a message from the user (multi-line text input).

Attribute: required, placeholder (prompt for user input).

<input type="file">: Allows users to upload files (multiple file types and files).

Attributes: accept (restricts file types), multiple (allows multiple files).

<input type="hidden">: Stores data that is not visible to the user (user ID).

Attribute: value (stores a value that is sent with the form but not visible).

<input type="submit">: Submits the form.

Attribute: value (sets the text of the button).

Create an HTML form that collects basic user details for registration. The form should include the following fields: Full Name (text input), Email Address (email input), Password (password input), Confirm Password (password input), Date of Birth (date input), Gender (radio buttons), and a checkbox for accepting Terms and Conditions. Implement validation such that: the Full Name and Email Address fields are required, the Password field must contain at least one uppercase letter, one digit, and one special character, and the Confirm Password field must match the Password field. The Date of Birth field should be a required field, and the user must check the Terms and Conditions checkbox before submitting the form.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>User Registration Form</title>
```

```
</head>
```

```
<body>
```

```
<h2>User Registration</h2>
```

```
<form action="/submit_registration" method="POST" id="registrationForm">
```

```
<!-- Full Name -->
```

```
<label for="fullName">Full Name:</label>
```

```
<input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">
```

```
<br><br>
```

```
<!-- Email Address -->
```

```
<label for="email">Email Address:</label>
```

```
<input type="email" id="email" name="email" required placeholder="Enter your email">
```

```
<br><br>
```

```
<!-- Password -->
```

```
<label for="password">Password:</label>
```

```
<input type="password" id="password" name="password" required placeholder="Enter your password"
```

```
  pattern=".*[A-Z].*" title="Password must contain at least one uppercase letter."
```

```
  minlength="8">
```

```
<br><br>
```

```
<!-- Confirm Password -->
```

```
<label for="confirmPassword">Confirm Password:</label>
```

<input type="password" id="confirmPassword" name="confirmPassword" required placeholder="Confirm your password">

<!-- Date of Birth -->

<label for="dob">Date of Birth:</label>

<input type="date" id="dob" name="dob" required>

<!-- Gender -->

<label>Gender:</label>

<input type="radio" id="male" name="gender" value="Male" required>

<label for="male">Male</label>

<input type="radio" id="female" name="gender" value="Female" required>

<label for="female">Female</label>

<input type="radio" id="other" name="gender" value="Other" required>

<label for="other">Other</label>

<!-- Terms and Conditions -->

<label>

<input type="checkbox" id="terms" name="terms" required>

I agree to the Terms and Conditions

</label>


```

        <!-- Submit Button -->

        <button type="submit">Submit</button>

    </form>

</script>

    // JavaScript to ensure password and confirm password match

    document.getElementById('registrationForm').onsubmit = function(event) {

        var password = document.getElementById('password').value;

        var confirmPassword = document.getElementById('confirmPassword').value;

        if (password !== confirmPassword) {

            alert("Passwords do not match.");

            event.preventDefault(); // Prevent form submission

        }

    };

</script>

</body>

</html>

```

Create a job application form with the following fields: Full Name (text input), Email Address (email input), Phone Number (tel input, valid 10-digit number), Position Applied For (dropdown list with at least 3 job positions), Skills (checkboxes, multiple selection allowed), Resume Upload (file input, only .pdf or .docx files allowed), and a Cover Letter (textarea, optional). Implement validation to ensure that all fields are properly validated: Phone Number must follow the correct pattern, and the Resume Upload field must accept only specified file types.

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Job Application Form</title>

</head>

<body>

  <h2>Job Application</h2>

  <form action="/submit_application" method="POST" enctype="multipart/form-data"
id="jobApplicationForm">

    <!-- Full Name -->

    <label for="fullName">Full Name:</label>

    <input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">

    <br><br>

    <!-- Email Address -->

    <label for="email">Email Address:</label>

    <input type="email" id="email" name="email" required placeholder="Enter your email">

    <br><br>

    <!-- Phone Number -->

    <label for="phone">Phone Number:</label>

    <input type="tel" id="phone" name="phone" required placeholder="Enter your phone number"
      pattern="\d{10}" title="Phone number must be exactly 10 digits">

    <br><br>

    <!-- Position Applied For -->

    <label for="position">Position Applied For:</label>

    <select id="position" name="position" required>
```

<option value="">Select a position</option>

<option value="developer">Software Developer</option>

<option value="designer">Web Designer</option>

<option value="manager">Project Manager</option>

</select>

<!-- Skills -->

<label for="skills">Skills:</label>

<input type="checkbox" id="html" name="skills" value="HTML">

<label for="html">HTML</label>

<input type="checkbox" id="css" name="skills" value="CSS">

<label for="css">CSS</label>

<input type="checkbox" id="javascript" name="skills" value="JavaScript">

<label for="javascript">JavaScript</label>

<input type="checkbox" id="python" name="skills" value="Python">

<label for="python">Python</label>

<input type="checkbox" id="java" name="skills" value="Java">

<label for="java">Java</label>

<!-- Resume Upload -->

<label for="resume">Resume (PDF or DOCX only):</label>

<input type="file" id="resume" name="resume" accept=".pdf,.docx" required>


```
<!-- Cover Letter -->

<label for="coverLetter">Cover Letter (optional):</label><br>

<textarea id="coverLetter" name="coverLetter" rows="4" cols="50" placeholder="Write your cover letter here (optional)"></textarea>

<br><br>

<!-- Submit Button -->

<button type="submit">Submit Application</button>

</form>

<script>

// JavaScript validation for file type and file size (optional)

document.getElementById('jobApplicationForm').onsubmit = function(event) {

    var fileInput = document.getElementById('resume');

    var filePath = fileInput.value;

    var allowedExtensions = /(\\.pdf|\\.docx)$/i;

    if (!allowedExtensions.exec(filePath)) {

        alert("Please upload a file with .pdf or .docx extension.");

        fileInput.value = "";

        event.preventDefault(); // Prevent form submission

    }

};

</script>

</body>

</html>
```

Design a feedback form that includes the following fields: Full Name (text input), Email Address (email input), Rating (radio buttons for a 1-5 star rating), Comments (textarea, optional). The Rating field must be required, and the Email Address field should follow proper email format validation. Ensure the Full Name field is required and implement error messages for incomplete or invalid fields.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Feedback Form</title>

</head>

<body>

  <h2>Feedback Form</h2>

  <form action="/submit_feedback" method="POST">

    <!-- Full Name -->

    <label for="fullName">Full Name:</label>

    <input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">

    <br><br>

    <!-- Email Address -->

    <label for="email">Email Address:</label>

    <input type="email" id="email" name="email" required placeholder="Enter your email">

    <br><br>

    <!-- Rating -->

    <label>Rating:</label><br>

    <input type="radio" id="rating1" name="rating" value="1" required>
```

<label for="rating1">1 Star</label>

<input type="radio" id="rating2" name="rating" value="2">

<label for="rating2">2 Stars</label>

<input type="radio" id="rating3" name="rating" value="3">

<label for="rating3">3 Stars</label>

<input type="radio" id="rating4" name="rating" value="4">

<label for="rating4">4 Stars</label>

<input type="radio" id="rating5" name="rating" value="5">

<label for="rating5">5 Stars</label>

<!-- Comments -->

<label for="comments">Comments (Optional):</label>

<textarea id="comments" name="comments" rows="4" placeholder="Share your feedback..."></textarea>

<!-- Submit Button -->

<button type="submit">Submit Feedback</button>

</form>

</body>

</html>

Develop a Contact Us form with Full Name (text input), Email Address (email input), Message (textarea, required), and Phone Number (tel input, optional, valid 10-digit number). Implement validation to ensure that the Full Name, Email Address, and Message fields are mandatory. Validate the Phone Number input field to ensure it only accepts a 10-digit phone number format using pattern matching.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Contact Us</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Contact Us</h2>
```

```
  <form action="/submit_contact" method="POST">
```

```
    <!-- Full Name -->
```

```
    <label for="fullName">Full Name:</label>
```

```
    <input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">
```

```
    <br><br>
```

```
    <!-- Email Address -->
```

```
    <label for="email">Email Address:</label>
```

```
    <input type="email" id="email" name="email" required placeholder="Enter your email">
```

```
    <br><br>
```

```
    <!-- Message -->
```

```
    <label for="message">Message:</label><br>
```

```
    <textarea id="message" name="message" rows="5" required placeholder="Enter your message..."></textarea>
```

```
<br><br>
```

```
<!-- Phone Number (Optional) -->
```

```
<label for="phone">Phone Number (Optional):</label>
```

```
<input type="tel" id="phone" name="phone" placeholder="Enter your phone number (10 digits)"
      pattern="^\d{10}$" title="Phone number must be exactly 10 digits">
```

```
<br><br>
```

```
<!-- Submit Button -->
```

```
<button type="submit">Submit</button>
```

```
</form>
```

```
</body>
```

```
</html>
```

Create an event registration form that includes Full Name (text input), Email Address (email input), Phone Number (tel input, optional), Preferred Session Time (dropdown list with time slots), Dietary Preferences (checkboxes for options like Vegetarian, Non-Vegetarian, Vegan), and Additional Comments (textarea, optional). The Full Name, Email Address, and Preferred Session Time fields should be required. Implement validation for the Phone Number field to ensure it contains a valid 10-digit number.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Event Registration Form</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Event Registration</h2>
```

```
  <form action="/submit_registration" method="POST">
```

<!-- Full Name -->

<label for="fullName">Full Name:</label>

<input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">

<!-- Email Address -->

<label for="email">Email Address:</label>

<input type="email" id="email" name="email" required placeholder="Enter your email">

<!-- Phone Number (Optional) -->

<label for="phone">Phone Number (Optional):</label>

<input type="tel" id="phone" name="phone" placeholder="Enter your phone number (10 digits)"
pattern="^\d{10}\$" title="Phone number must be exactly 10 digits">

<!-- Preferred Session Time -->

<label for="sessionTime">Preferred Session Time:</label>

<select id="sessionTime" name="sessionTime" required>

<option value="">Select a time slot</option>

<option value="morning">Morning (9:00 AM - 12:00 PM)</option>

<option value="afternoon">Afternoon (1:00 PM - 4:00 PM)</option>

<option value="evening">Evening (5:00 PM - 8:00 PM)</option>

</select>


```
<!-- Dietary Preferences -->

<label>Dietary Preferences:</label><br>

<input type="checkbox" id="vegetarian" name="dietary[]" value="Vegetarian">

<label for="vegetarian">Vegetarian</label><br>

<input type="checkbox" id="nonVegetarian" name="dietary[]" value="Non-Vegetarian">

<label for="nonVegetarian">Non-Vegetarian</label><br>

<input type="checkbox" id="vegan" name="dietary[]" value="Vegan">

<label for="vegan">Vegan</label><br>

<br>

<!-- Additional Comments -->

<label for="comments">Additional Comments (Optional):</label><br>

<textarea id="comments" name="comments" rows="4" placeholder="Enter any additional
comments..."></textarea>

<br><br>

<!-- Submit Button -->

<button type="submit">Register</button>

</form>

</body>

</html>
```

Build a survey form that collects the following information: Full Name (text input), Age (number input, between 18 and 100), Gender (radio buttons: Male, Female, Other), Preferred Programming Language (multiple select dropdown), and Feedback (textarea). Implement validation for the Age field to accept only numbers between 18 and 100, ensure that at least one option is selected for the Preferred Programming Language, and make the Full Name, Age, and Gender fields required.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Survey Form</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Survey Form</h2>
```

```
  <form action="/submit_survey" method="POST">
```

```
    <!-- Full Name -->
```

```
    <label for="fullName">Full Name:</label>
```

```
    <input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">
```

```
    <br><br>
```

```
    <!-- Age -->
```

```
    <label for="age">Age:</label>
```

```
    <input type="number" id="age" name="age" required min="18" max="100" placeholder="Enter your age">
```

```
    <br><br>
```

```
    <!-- Gender -->
```

```
    <label>Gender:</label><br>
```

```
    <input type="radio" id="male" name="gender" value="Male" required>
```

```
    <label for="male">Male</label>
```

<input type="radio" id="female" name="gender" value="Female">

<label for="female">Female</label>

<input type="radio" id="other" name="gender" value="Other">

<label for="other">Other</label>

<!-- Preferred Programming Language -->

<label for="language">Preferred Programming Language:</label>

<select id="language" name="language[]" multiple required>

<option value="JavaScript">JavaScript</option>

<option value="Python">Python</option>

<option value="Java">Java</option>

<option value="C++">C++</option>

<option value="Ruby">Ruby</option>

</select>

<!-- Feedback -->

<label for="feedback">Feedback:</label>

<textarea id="feedback" name="feedback" rows="4" placeholder="Provide your feedback..."></textarea>

<!-- Submit Button -->

<button type="submit">Submit Survey</button>

</form>

</body>

</html>

Create a newsletter subscription form that includes Full Name (text input), Email Address (email input), Subscription Frequency (radio buttons for Weekly, Monthly), and Receive Offers (checkbox). Validate that the Full Name and Email Address fields are required. Ensure the Email Address is in the correct email format and that the Subscription Frequency field has one option selected. Implement error handling for missing or incorrect data.

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Newsletter Subscription</title>

</head>

<body>

<h2>Newsletter Subscription</h2>

<form action="/submit_subscription" method="POST">

<!-- Full Name -->

<label for="fullName">Full Name:</label>

<input type="text" id="fullName" name="fullName" required placeholder="Enter your full name">

<!-- Email Address -->

<label for="email">Email Address:</label>

<input type="email" id="email" name="email" required placeholder="Enter your email address">

<!-- Subscription Frequency -->

<label>Subscription Frequency:</label>

<input type="radio" id="weekly" name="frequency" value="Weekly" required>

<label for="weekly">Weekly</label>

<input type="radio" id="monthly" name="frequency" value="Monthly">

<label for="monthly">Monthly</label>

<!-- Receive Offers -->

<label for="offers">Receive Offers:</label>

<input type="checkbox" id="offers" name="offers">

<label for="offers">Yes, I want to receive special offers.</label>

<!-- Submit Button -->

<button type="submit">Subscribe</button>

</form>

</body>

</html>

HTML Tables:

In HTML, tables are used to represent tabular data in a structured format consisting of rows and columns. They are created using the `<table>` tag and consist of various components such as table rows (`<tr>`), table headers (`<th>`), and table data cells (`<td>`). Tables are essential for organizing and displaying data clearly, especially when dealing with large sets of data like spreadsheets, lists, or reports.

Basic syntax:

```
<table>

  <tr>

    <th>Header 1</th>

    <th>Header 2</th>

    <th>Header 3</th>

  </tr>

  <tr>

    <td>Data 1</td>

    <td>Data 2</td>

    <td>Data 3</td>

  </tr>

</table>
```

Example

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Employee Information </title>

  <style>

    /* Basic table styling */

    table {

      width: 80%;

      margin: 20px auto;
```

```
border-collapse: collapse; /* Removes double borders */
}

/* Styling for table header and data cells */
th, td {
    padding: 12px; /* Adds space inside cells */
    text-align: center; /* Centers text in cells */
    border: 1px solid #ddd; /* Adds border around cells */
}

/* Styling for table header */
th {
    background-color: #f2f2f2; /* Light gray background for header */
}

/* Adds alternating row colors for better readability */
tr:nth-child(even) {
    background-color: #f9f9f9; /* Light background for even rows */
}

/* Adds background color for the table caption */
caption {
    font-size: 1.5em; /* Increases font size for the caption */
    margin: 10px 0; /* Adds margin above and below the caption */
}

</style>
</head>
<body>

<h2>Employee Information</h2>
```

<!-- Table with a header, body, and footer -->

<table>

<!-- Caption for the table -->

<caption>Company Employee Data</caption>

<!-- Table Header Section -->

<thead>

<tr>

<th>ID</th> <!-- Table header for the ID column -->

<th>Name</th> <!-- Table header for the Name column -->

<th>Position</th> <!-- Table header for the Position column -->

<th>Salary</th> <!-- Table header for the Salary column -->

</tr>

</thead>

<!-- Table Body Section -->

<tbody>

<!-- Row 1 -->

<tr>

<td>1</td> <!-- Data cell for employee ID -->

<td>Hari Prasad</td> <!-- Data cell for employee Name (Nepali name in English) -->

>

<td>Manager</td> <!-- Data cell for employee Position -->

<td>\$80,000</td> <!-- Data cell for employee Salary -->

</tr>

<!-- Row 2 -->

<tr>

<td>2</td>

<td>Shanti Devi</td>

<td>Software Engineer</td>

```
<td>$70,000</td>
</tr>
<!-- Row 3 -->
<tr>
  <td>3</td>
  <td>Ram Bahadur</td>
  <td>Graphic Designer</td>
<
```

1. Introduction to HTML Forms

HTML forms are essential components of web applications, providing a structured means for users to submit data to a server for processing. Forms are integral in user interaction with websites, particularly for tasks such as data collection, user registration, and authentication. Through the use of various form controls, web developers can design flexible and interactive web forms that cater to diverse user needs.

A form in HTML is created using the `<form>` tag, which acts as a container for input elements. These input elements, such as `<input>`, `<textarea>`, `<select>`, and `<button>`, allow users to enter or select data which is then submitted for processing.

2. Structure of an HTML Form

An HTML form is structured by several key components:

- **<form> Tag:** The form element is used to encapsulate the form's input elements and define its submission behavior.
- **Form Attributes:**
 - **action:** Specifies the URL or endpoint where the form data will be sent for processing after submission.
 - **method:** Specifies the HTTP method (either GET or POST) to be used when sending the form data to the server.

Basic Syntax:

```
<form action="submit_form.php" method="POST">
  <!-- Form elements go here -->
</form>
```

- **action:** Refers to the server-side script or resource that will handle the form submission.
- **method:** Defines the method for transmitting the form data. The POST method is commonly used for sensitive or large data, while GET is typically used for non-sensitive or small queries.

3. Common HTML Form Elements

a. <input> Element

The <input> element is one of the most widely used form elements. It can represent different types of input fields depending on the value assigned to the type attribute. This element facilitates user interaction with the form by accepting various types of data such as text, numbers, dates, passwords, etc.

- **Common type values:**
 - type="text": A basic text field for single-line input.
 - type="password": A field for entering passwords, where characters are obscured.
 - type="radio": Radio buttons, which allow the user to select one option from a set.
 - type="checkbox": Checkboxes, which enable users to select multiple options.
 - type="submit": A button that submits the form.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="user_name" required>
```

- **required:** Ensures the field must be filled before submission.

b. <textarea> Element

The <textarea> element is utilized for multi-line text input, providing users with a larger input area to enter detailed information or comments.

Example:

```
<label for="message">Message:</label>
<textarea id="message" name="user_message" rows="4" cols="50" required></textarea>
```

- **rows="4":** Specifies the number of visible lines in the text area.
- **cols="50":** Specifies the number of visible characters per line.

c. Radio Buttons

Radio buttons are used when the user must select one option from a predefined set. When radio buttons are grouped under the same name attribute, selecting one button automatically deselects the others.

Example:

```
<label for="male">Male</label>
<input type="radio" id="male" name="gender" value="male">
<label for="female">Female</label>
<input type="radio" id="female" name="gender" value="female">
```

- **name="gender"**: Groups the radio buttons, ensuring only one can be selected.

d. Checkboxes

Checkboxes allow users to select multiple options from a list. Unlike radio buttons, checkboxes allow multiple selections.

Example:

```
<label for="newsletter">Subscribe to Newsletter:</label>
<input type="checkbox" id="newsletter" name="newsletter" value="yes">
```

e. <select> and <option> Elements

The <select> element is used to create a dropdown menu, while the <option> element defines the individual options within the dropdown.

Example:

```
<label for="country">Country:</label>
<select id="country" name="country">
  <option value="USA">USA</option>
  <option value="Canada">Canada</option>
  <option value="India">India</option>
</select>
```

f. Submit Button

A submit button, defined by the <button> tag with the type="submit" attribute, is used to send the form data to the server for processing.

Example:

```
<button type="submit">Submit</button>
```

4. Methods for Form Submission

HTML forms can employ two primary methods to send data to the server: GET and POST. The choice of method determines how form data is transmitted and the visibility of the data.

1. GET Method:

- Data is appended to the URL in the form of query parameters.
- Suitable for non-sensitive data, such as search queries or filtering options.
- Example:

2. `<form action="search_results.php" method="GET">`

3. `<input type="text" name="query" required>`

4. `<button type="submit">Search</button>`

5. `</form>`

6. POST Method:

- Data is sent in the body of the HTTP request, ensuring it is not visible in the URL.
- Suitable for sensitive data, such as passwords, or for large datasets.
- Example:

7. `<form action="submit_form.php" method="POST">`

8. `<input type="text" name="name" required>`

9. `<input type="password" name="password" required>`

10. `<button type="submit">Login</button>`

11. `</form>`

HTML Contact Form

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Contact Form</title>
```

```
<style>
```

```
/* Basic form styling */
```

```
form {
```

```
    width: 50%;
```

```
    margin: auto;
```

```
    padding: 20px;
```

```
    background-color: #f2f2f2;
```

```
    border-radius: 5px;
```

```
}
```

```
label {
```

```
    display: block;
```

```
    margin-bottom: 10px;
```

```
    font-weight: bold;
```

```
}
```

```
input, textarea, select, button {
```

```
    width: 100%;
```

```
    padding: 10px;
```

```
    margin-bottom: 20px;
```

```
    border: 1px solid #ccc;
```

```
    border-radius: 5px;
```

```

    }
    button {
        background-color: #4CAF50;
        color: white;
        font-size: 16px;
    }
</style>
</head>
<body>

<h2>Contact Us</h2>

<form action="submit_form.php" method="POST">
    <!-- Name input -->
    <label for="name">Name:</label>
    <input type="text" id="name" name="user_name" required>

    <!-- Email input -->
    <label for="email">Email:</label>
    <input type="email" id="email" name="user_email" required>

    <!-- Message input -->
    <label for="message">Message:</label>
    <textarea id="message" name="user_message" rows="4" required></textarea>

    <!-- Gender selection -->
    <label>Gender:</label>
    <input type="radio" id="male" name="gender" value="male"> <label
for="male">Male</label>
    <input type="radio" id="female" name="gender" value="female"> <label
for="female">Female</label>

    <!-- Country dropdown -->
    <label for="country">Country:</label>
    <select id="country" name="country">
        <option value="USA">USA</option>
        <option value="Canada">Canada</option>
        <option value="India">India</option>
    </select>

    <!-- Newsletter subscription -->
    <label for="subscribe">
        <input type="checkbox" id="subscribe" name="subscribe" value="yes"> Subscribe to
newsletter
    </label>

    <!-- Submit button -->

```

```
    <button type="submit">Submit</button>
</form>
```

```
</body>
</html>
```

What is an HTML Table?

An **HTML table** is a way of displaying data in a two-dimensional grid made up of rows and columns. It is used to represent complex data in a clear and organized manner, with each row representing a single data record and each column corresponding to an attribute or property of that data.

Tables are typically used when the data is interrelated, such as when showing numerical values, financial data, schedules, or contact information. Unlike lists or paragraphs, which are linear, tables provide a more structured view of data, making it easier to compare and contrast information across different categories.

Structure of an HTML Table

An HTML table consists of several key components that define its structure. These components include the table itself, rows, columns, headers, data cells, and optionally, captions and footers

1. <table> - The Table Element

- The **<table>** element is the outermost tag used to define a table.
- This tag serves as the container for the table's entire structure, including rows, columns, headers, and data.
- It groups all table elements together into one logical structure.

```
<table>  
  <!-- Table content goes here -->  
</table>
```

2. <caption> - The Table Title

- The **<caption>** element provides a title or description for the table.
- This is an optional element, but it is highly recommended for accessibility and to provide a clear understanding of the table's content.
- The caption typically appears above the table.
- It helps users quickly identify the purpose of the table.

```
<caption>Employee Information</caption>
```

3. <thead> - Table Header Section

- The **<thead>** element is used to group the header content in a table.
- It usually contains a row of headings that describe the data in the columns beneath them.
- The **<thead>** section is placed at the top of the table, before the table body.
- It is often used to define column titles, and the text inside these cells is generally bold and centered by default.

```
<thead>  
  <tr>  
    <th>Name</th>  
    <th>Position</th>  
    <th>Salary</th>  
  </tr>
```

</thead>

4. <tr> - Table Row

- The <tr> element defines a row within the table.
- Each row can contain one or more <td> (data) or <th> (header) cells, depending on whether it is a header row or data row.
- A row groups cells in a horizontal line, and multiple rows are placed one after the other to create the table's grid structure.

```
<tr>
  <td>Ram Syaham</td>
  <td>Manager</td>
  <td>NRS 220,000</td>
</tr>
```

5. <th> - Table Header Cell

- The <th> element defines a table header cell, which is used to represent column or row headers.
- Text inside <th> cells is typically bold and centered by default.
- <th> tags help clarify what each column represents, making the table easier to understand.

```
<th>Name</th>
<th>Position</th>
<th>Salary</th>
```

6. <td> - Table Data Cell

- The <td> element defines a standard data cell in the table.
- This tag holds the actual content or data for a specific column in a row.
- <td> tags are used for displaying values such as text, numbers, links, or even images within a table.

```
<td>Ram Syaham</td>
<td>Manager</td>
<td>NRS 220,000</td>
```

7. <tbody> - Table Body Section

- The <tbody> element is used to group the body content of the table.
- It is commonly used to wrap the rows that contain the data.
- This section is important for organizing the data content separately from the header and footer sections.
- This element is optional, but it enhances readability and provides better structure for the table, especially when dealing with complex or large data sets.

```
<tbody>
  <tr>
    <td>Ram Syaham</td>
```

```

        <td>Manager</td>
        <td>NRS 220,000</td>
    </tr>
    <tr>
        <td>Sita Kumari</td>
        <td>Developer</td>
        <td>NRS 150,000</td>
    </tr>
</tbody>

```

8. <tfoot> - Table Footer Section

- The <tfoot> element is used to group the footer content of the table.
- Typically, it contains summary data or information that is relevant to the table as a whole, such as totals, averages, or general notes.
- Like <thead>, the <tfoot> element is used for grouping, and it often appears at the bottom of the table, although its placement is flexible.

```

<tfoot>
<tr>
    <td colspan="3">Total Salary: NRS 370,000</td>
</tr>
</tfoot>

```

Basic Example of an HTML Table

```

<table border="1" cellpadding="10" cellspacing="5">
    <caption>Employee Information</caption>
    <thead>
        <tr>
            <th>Name</th>
            <th>Position</th>
            <th>Salary</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Ram Syaham</td>
            <td>Manager</td>
            <td>NRS 220,000</td>
        </tr>
        <tr>
            <td>Sita Kumari</td>
            <td>Developer</td>
            <td>NRS 150,000</td>
        </tr>
    </tbody>
</table>

```

```
<tr>
  <td colspan="3">Total Salary: NRS 370,000</td>
</tr>
</tfoot>
</table>
```

Table Alignment

- Alignment is a critical aspect of presenting data effectively in tables.
- In HTML, alignment determines how the entire table, individual rows, and the content of table cells are positioned relative to the surrounding content.
- HTML provides several attributes that enable basic alignment for both horizontal and vertical positioning of content.

1. Aligning the Entire Table

- To align an entire HTML table within its container, the **align** attribute is used within the `<table>` tag. This attribute is deprecated in modern HTML, but it is still widely supported in legacy browsers.

Values of the align Attribute:

- **left:** Aligns the table to the left side of the container.
- **center:** Centers the table within the container.
- **right:** Aligns the table to the right side of the container.

Example:

```
<table align="center">
  <caption>Employee Information</caption>
  <thead>
    <tr>
      <th>Name</th>
      <th>Position</th>
      <th>Salary</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ram Syaham</td>
      <td>Manager</td>
      <td>NRS 220,000</td>
    </tr>
    <tr>
      <td>Sita Kumari</td>
      <td>Developer</td>
      <td>NRS 150,000</td>
    </tr>
  </tbody>
</table>
```

```
</tbody>
</table>
```

2. Aligning Table Cells Horizontally

The **align** attribute can also be used within individual table cells, specifically the `<th>` (header) and `<td>` (data) tags, to control the horizontal alignment of the content. This allows for alignment of content within a column.

Values of the align Attribute for Cells:

- **left:** Aligns the content to the left of the cell.
- **center:** Centers the content within the cell.
- **right:** Aligns the content to the right of the cell.

Example:

```
<table>
  <thead>
    <tr>
      <th align="center">Name</th>
      <th align="left">Position</th>
      <th align="right">Salary</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td align="center">Ram Syaham</td>
      <td align="left">Manager</td>
      <td align="right">NRS 220,000</td>
    </tr>
    <tr>
      <td align="center">Sita Kumari</td>
      <td align="left">Developer</td>
      <td align="right">NRS 150,000</td>
    </tr>
  </tbody>
</table>
```

3. Aligning Table Cells Vertically

Vertical alignment in HTML tables refers to positioning the content of cells along the vertical axis. This can be controlled using the **valign** attribute within `<td>` or `<th>` tags.

Values of the valign Attribute:

- **top:** Aligns the content to the top of the cell.
- **middle:** Aligns the content in the middle of the cell.
- **bottom:** Aligns the content to the bottom of the cell.

Example:

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Position</th>
      <th>Salary</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td valign="top">Ram Syaham</td>
      <td valign="middle">Manager</td>
      <td valign="bottom">NRS 220,000</td>
    </tr>
    <tr>
      <td valign="top">Sita Kumari</td>
      <td valign="middle">Developer</td>
      <td valign="bottom">NRS 150,000</td>
    </tr>
  </tbody>
</table>
```

4. Aligning Entire Rows

Although HTML does not provide a direct attribute for aligning entire rows, you can use the **align** attribute within the <tr> tag. This will apply the alignment to all cells within the row.

Example:

```
<table>
  <thead>
    <tr align="center">
      <th>Name</th>
      <th>Position</th>
      <th>Salary</th>
    </tr>
  </thead>
  <tbody>
    <tr align="right">
      <td>Ram Syaham</td>
      <td>Manager</td>
      <td>NRS 220,000</td>
    </tr>
    <tr align="left">
      <td>Sita Kumari</td>
      <td>Developer</td>
```

```
<td>NRS 150,000</td>
</tr>
</tbody>
</table>
```

HTML Table Attributes

HTML tables can be customized using various attributes to control their appearance and behavior. These attributes are used to define the table’s structure, style, alignment, and spacing.

Attribute	Description	Example	Code
border	Specifies the width of the table's border.	Defines the border thickness of the table.	<code><table border="1"></code>
cellspacing	Defines the space between individual table cells.	Adds space between cells in the table.	<code><table cellspacing="5"></code>
cellpadding	Sets the space between the content of a cell and its border.	Adds padding inside the cells to create more space around content.	<code><table cellpadding="10"></code>
width	Sets the width of the entire table or its columns.	Defines the width of the table in percentage or pixels.	<code><table width="80%"></code>
height	Specifies the height of the entire table.	Defines the height of the table in pixels.	<code><table height="200px"></code>
align	Aligns the table within its container (left, center, right).	Specifies the alignment of the table.	<code><table align="center"></code>
bgcolor	Sets the background color of the entire table.	Specifies a background color for the table.	<code><table bgcolor="#f0f0f0"></code>
id	Defines a unique identifier for the table, useful for styling and scripts.	Assigns an ID to the table for CSS or JavaScript manipulation.	<code><table id="employeeTable"></code>
class	Assigns a class name to the table, for CSS styling or JavaScript targeting.	Allows CSS or JavaScript to target the table.	<code><table class="employee-list"></code>
style	Defines inline CSS styles to be applied directly to the table.	Allows custom CSS styling directly within the table tag.	<code><table style="border:1px solid black; width: 100%;"></code>

Spanning Multiple Rows or Columns

HTML provides the **colspan** and **rowspan** attributes to allow cells to span multiple columns or rows, respectively. These attributes are used to create complex table structures, which can be useful for displaying summary data or for organizing related content.

- **colspan**: Specifies how many columns a cell should span.
- **rowspan**: Specifies how many rows a cell should span.

Example with colspan:

```

<table border="1">
  <tr>
    <th colspan="2">Employee Info</th>
    <th rowspan="2">Salary</th>
  </tr>
  <tr>
    <td>Ram Syaham</td>
    <td>Manager</td>
  </tr>
  <tr>
    <td>Sita Kumari</td>
    <td>Developer</td>
    <td>NRS 150,000</td>
  </tr>
</table>

```

Combining colspan and rowspan

You can combine both **colspan** and **rowspan** in the same table to create more complex table layouts.

Example:

```

<table border="1">
  <tr>
    <th colspan="3">Employee Information</th>
  </tr>
  <tr>
    <td rowspan="2">Ram Syaham</td>
    <td>Manager</td>
    <td>NRS 220,000</td>
  </tr>
  <tr>
    <td>Developer</td>
    <td>NRS 150,000</td>
  </tr>
</table>

```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Employee Table</title>
</head>
<body>
  <table id="employeeTable" class="employee-list" border="1" cellspacing="5" cellpadding="10"
width="80%" height="300px" align="center" bgcolor="#f0f0f0" summary="This table displays employee
details including name, position, and salary.">
    <caption>Employee Salary Data</caption>
    <thead>
      <tr>
        <th scope="col">Employee Name</th>
        <th scope="col">Position</th>
        <th scope="col">Salary (NRS)</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>Ram Syaham</td>
        <td>Manager</td>
        <td>NRS 220,000</td>
      </tr>
      <tr>
        <td>Sita Sharma</td>
        <td>Developer</td>
        <td>NRS 180,000</td>
      </tr>
      <tr>
        <td>Hari Kumar</td>
        <td>Designer</td>
        <td>NRS 150,000</td>
      </tr>
      <tr>
        <td>Pooja Pandey</td>
        <td>HR</td>
        <td>NRS 140,000</td>
      </tr>
    </tbody>
    <tfoot>
      <tr>
        <td colspan="2" align="right"><strong>Total Salary</strong></td>
        <td><strong>NRS 690,000</strong></td>
      </tr>
    </tfoot>
  </table>
</body>
</html>

```

1. Introduction to HTML Forms

HTML forms are essential components of web applications, providing a structured means for users to submit data to a server for processing. Forms are integral in user interaction with websites, particularly for tasks such as data collection, user registration, and authentication. Through the use of various form controls, web developers can design flexible and interactive web forms that cater to diverse user needs.

A form in HTML is created using the `<form>` tag, which acts as a container for input elements. These input elements, such as `<input>`, `<textarea>`, `<select>`, and `<button>`, allow users to enter or select data which is then submitted for processing.

2. Structure of an HTML Form

An HTML form is structured by several key components:

- **`<form>` Tag:** The form element is used to encapsulate the form's input elements and define its submission behavior.
- **Form Attributes:**
 - **action:** Specifies the URL or endpoint where the form data will be sent for processing after submission.
 - **method:** Specifies the HTTP method (either GET or POST) to be used when sending the form data to the server.

Basic Syntax:

```
<form action="submit_form.php" method="POST">
  <!-- Form elements go here -->
</form>
```

- **action:** Refers to the server-side script or resource that will handle the form submission.
- **method:** Defines the method for transmitting the form data. The POST method is commonly used for sensitive or large data, while GET is typically used for non-sensitive or small queries.

3. Common HTML Form Elements

a. `<input>` Element

The `<input>` element is one of the most widely used form elements. It can represent different types of input fields depending on the value assigned to the type attribute. This element facilitates user interaction with the form by accepting various types of data such as text, numbers, dates, passwords, etc.

- **Common type values:**
 - `type="text"`: A basic text field for single-line input.
 - `type="password"`: A field for entering passwords, where characters are obscured.
 - `type="radio"`: Radio buttons, which allow the user to select one option from a set.
 - `type="checkbox"`: Checkboxes, which enable users to select multiple options.
 - `type="submit"`: A button that submits the form.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="user_name" required>
```

- **required:** Ensures the field must be filled before submission.

b. <textarea> Element

The <textarea> element is utilized for multi-line text input, providing users with a larger input area to enter detailed information or comments.

Example:

```
<label for="message">Message:</label>
<textarea id="message" name="user_message" rows="4" cols="50" required></textarea>
```

- **rows="4"**: Specifies the number of visible lines in the text area.
- **cols="50"**: Specifies the number of visible characters per line.

c. Radio Buttons

Radio buttons are used when the user must select one option from a predefined set. When radio buttons are grouped under the same name attribute, selecting one button automatically deselects the others.

Example:

```
<label for="male">Male</label>
<input type="radio" id="male" name="gender" value="male">
<label for="female">Female</label>
<input type="radio" id="female" name="gender" value="female">
```

- **name="gender"**: Groups the radio buttons, ensuring only one can be selected.

d. Checkboxes

Checkboxes allow users to select multiple options from a list. Unlike radio buttons, checkboxes allow multiple selections.

Example:

```
<label for="newsletter">Subscribe to Newsletter:</label>
<input type="checkbox" id="newsletter" name="newsletter" value="yes">
```

e. <select> and <option> Elements

The <select> element is used to create a dropdown menu, while the <option> element defines the individual options within the dropdown.

Example:

```
<label for="country">Country:</label>
<select id="country" name="country">
  <option value="USA">USA</option>
  <option value="Canada">Canada</option>
```

```
<option value="India">India</option>
</select>
```

f. Submit Button

A submit button, defined by the `<button>` tag with the `type="submit"` attribute, is used to send the form data to the server for processing.

Example:

```
<button type="submit">Submit</button>
```

4. Methods for Form Submission

HTML forms can employ two primary methods to send data to the server: GET and POST. The choice of method determines how form data is transmitted and the visibility of the data.

1. GET Method:

- Data is appended to the URL in the form of query parameters.
- Suitable for non-sensitive data, such as search queries or filtering options.
- Example:

```
2. <form action="search_results.php" method="GET">
```

```
3.   <input type="text" name="query" required>
```

```
4.   <button type="submit">Search</button>
```

```
5. </form>
```

6. POST Method:

- Data is sent in the body of the HTTP request, ensuring it is not visible in the URL.
- Suitable for sensitive data, such as passwords, or for large datasets.
- Example:

```
7. <form action="submit_form.php" method="POST">
```

```
8.   <input type="text" name="name" required>
```

```
9.   <input type="password" name="password" required>
```

```
10.  <button type="submit">Login</button>
```

```
11. </form>
```

HTML Contact Form

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Contact Form</title>
```

```
  <style>
```

```
    /* Basic form styling */
```

```
    form {
```

```
      width: 50%;
```

```
      margin: auto;
```

```

padding: 20px;
background-color: #f2f2f2;
border-radius: 5px;
}
label {
display: block;
margin-bottom: 10px;
font-weight: bold;
}
input, textarea, select, button {
width: 100%;
padding: 10px;
margin-bottom: 20px;
border: 1px solid #ccc;
border-radius: 5px;
}
button {
background-color: #4CAF50;
color: white;
font-size: 16px;
}
</style>
</head>
<body>

<h2>Contact Us</h2>

<form action="submit_form.php" method="POST">
  <!-- Name input -->
  <label for="name">Name:</label>
  <input type="text" id="name" name="user_name" required>

  <!-- Email input -->
  <label for="email">Email:</label>
  <input type="email" id="email" name="user_email" required>

  <!-- Message input -->
  <label for="message">Message:</label>
  <textarea id="message" name="user_message" rows="4" required></textarea>

  <!-- Gender selection -->
  <label>Gender:</label>
  <input type="radio" id="male" name="gender" value="male"> <label for="male">Male</label>
  <input type="radio" id="female" name="gender" value="female"> <label for="female">Female</label>

  <!-- Country dropdown -->
  <label for="country">Country:</label>
  <select id="country" name="country">

```

```
<option value="USA">USA</option>
<option value="Canada">Canada</option>
<option value="India">India</option>
</select>
```

```
<!-- Newsletter subscription -->
```

```
<label for="subscribe">
```

```
  <input type="checkbox" id="subscribe" name="subscribe" value="yes"> Subscribe to newsletter
</label>
```

```
<!-- Submit button -->
```

```
<button type="submit">Submit</button>
```

```
</form>
```

```
</body>
```

```
</html>
```

Imagemaps

- An **imagemap** is a web design technique that allows an image to be subdivided into multiple regions, each of which can be linked to a different destination.
 - It is a method of embedding interactive features within an image, enabling specific parts of the image to function as hyperlinks.
 - When a user clicks on a designated area of the image, the browser will navigate to the specified URL, making imagemaps particularly useful for maps, diagrams, product images, or any visual representation where different sections need to be interactive.
 - Imagemaps are used in a variety of applications, such as clickable maps, image-based navigation menus, and detailed product images on e-commerce websites.
 - Imagemaps can be categorized into two main types:
 1. **Client-Side Imagemaps:** The click processing is handled directly by the browser, with coordinates sent to the browser for navigation.
 2. **Server-Side Imagemaps:** The coordinates of the clicked region are sent to a server, which processes them and determines the appropriate response or content.
- 1) **Client-Side Imagemaps:**
- A **Client-Side Imagemap** is an HTML technique that allows specific areas of an image to act as clickable links.
 - The clickable regions are defined using the `<map>` and `<area>` tags, and the browser processes the clicks directly without requiring server-side interaction.
 - It is commonly used for creating interactive images like maps, diagrams, and image-based navigation menus.

Basic Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Client-Side Imagemap</title>
</head>
<body>
  <h2>Client-Side Imagemap Example</h2>
  

  <map name="worldmap">
    <area shape="rect" coords="34,44,270,350" alt="USA" href="https://www.usa.gov">
    <area shape="circle" coords="400,200,50" alt="Brazil" href="https://www.brazil.gov.br">
    <area shape="poly" coords="100,150,200,150,250,250" alt="Canada" href="https://www.canada.ca">
  </map>
</body>
</html>
```

- 1) **<map>**: The `<map>` element defines the imagemap and links it to the image through the `usemap` attribute. The `name` attribute identifies the map.
 - a. **Example:**`<map name="worldmap">`
- 2) **<area>**: The `<area>` element defines each clickable region in the image.
 - a. **shape**: Specifies the shape of the clickable region. The common shapes are:
 - i. **rect**: A rectangular region.
 - ii. **circle**: A circular region.
 - iii. **poly**: A polygonal region.
 - b. **coords**: Provides the coordinates for the shape.
 - i. For **rect**: `coords="x1,y1,x2,y2"` (top-left and bottom-right corners).
 - ii. For **circle**: `coords="x,y,radius"` (center and radius).
 - iii. For **poly**: `coords="x1,y1,x2,y2,x3,y3..."` (set of points for the polygon).
 - c. **alt**: Describes the region for accessibility. This is important for users with visual impairments or when the image fails to load.
 - d. **href**: Defines the link that the user is redirected to when the region is clicked.

How Client-Side Imagemaps Work?

1. **Image Setup**: The `<img>` element links to the imagemap via the `usemap` attribute, which references the `<map>` element containing the clickable regions.

Example:

```

```

2. **Defining Clickable Areas**: The `<map>` element defines the clickable regions using the `<area>` tag. The `shape`, `coords`, and `href` attributes specify the region's shape (rectangle, circle, polygon), its coordinates, and the destination URL.

Example:

```
<map name="mapname">  
  <area shape="rect" coords="34,44,270,350" href="https://example.com">  
</map>
```

3. **User Interaction**: When a user clicks on a defined area, the browser compares the click's coordinates with the predefined regions. If a match is found, the browser navigates to the linked URL specified in the `href` attribute.
4. **Client-Side Processing**: All processing (coordinate checking and navigation) is done on the client-side, ensuring fast and efficient user interaction without needing server communication.

Advantages of Client-Side Imagemaps

- 1) **No Server-Side Processing**: Client-side imagemap do not require any server-side interaction, making them fast and efficient. All the processing happens directly in the user's browser.
- 2) **Interactive and User-Friendly**: Client-side imagemap allow users to interact with images, improving user experience and making the website more engaging.

- 3) **Instant Response:** Since all actions are processed locally on the client-side, the response time is immediate, providing a seamless and smooth user experience.
- 4) **Accessibility:** By using the alt attribute for each clickable region, client-side imagemaps can be made accessible to users with visual impairments, improving the overall accessibility of the website.

Limitations of Client-Side Imagemaps

- 1) **Responsiveness Issues:** Fixed pixel coordinates may cause misalignment when the image is resized, especially on devices with different screen sizes, making the imagemap less effective for responsive design.
- 2) **Limited Functionality:** Client-side imagemaps are static and cannot handle dynamic interactions that require server-side processing, such as content generation or database queries.
- 3) **SEO Concerns:** Since the links in an imagemap are embedded in an image, search engines may not index them as effectively as traditional hyperlinks, which could impact the SEO of the webpage.
- 4) **Dependence on Image Load:** If the image fails to load or is slow to load, the clickable areas will not be functional, rendering the imagemap ineffective.

Best Practices for Client-Side Imagemaps

- 1) **Use Descriptive alt Text:** Ensure that each clickable area in the imagemap has a meaningful alt attribute to enhance accessibility for users with disabilities, as well as improve the user experience when the image fails to load.
- 2) **Ensure Proper Image Size:** Always use the image at its original size or implement responsive design techniques to ensure the clickable areas remain correctly aligned across different screen sizes and devices.
- 3) **Test on Multiple Devices:** Since client-side imagemaps may have alignment issues on various screen sizes, test them on different devices to ensure the clickable areas are functioning properly.
- 4) **Combine with JavaScript:** For more complex interactions (e.g., dynamically changing content or advanced user actions), JavaScript can be used to enhance or modify the behavior of the imagemap after a region is clicked.

2.Server-Side Imagemaps

- A **Server-Side Imagemap** is a technique in web development that facilitates interaction with specific areas of an image by clicking on them.
- Unlike **Client-Side Imagemaps**, where all interactions are processed within the client's browser, Server-Side Imagemaps necessitate communication with the web server to process the click event and determine the appropriate action.
- This technique is useful when complex logic, dynamic content generation, or database queries are required upon user interaction.
- Server-Side Imagemaps enable a highly interactive experience, providing the ability to handle more advanced tasks, such as retrieving data from a server or performing complex calculations based on the user's input.

How Server-Side Imagemaps works?

1. Image Setup:

The tag is used to display the image on which the user will click. The image is wrapped inside an HTML form, enabling the collection of click coordinates and their transmission to the server. The coordinates represent the specific region of the image that the user interacts with.

Example:

```
<form action="process_imagemap.php" method="POST">
  
  <input type="hidden" name="x" id="x">
  <input type="hidden" name="y" id="y">
  <input type="submit" value="Submit">
</form>
```

2. Capturing Click Coordinates:

When a user clicks on a particular region of the image, the coordinates of that click are captured using JavaScript. These coordinates (the x and y values) are then inserted into hidden form fields, which are submitted along with the form.

Example (JavaScript):

```
document.querySelector('img').addEventListener('click', function(event) {
  var rect = event.target.getBoundingClientRect();
  var x = event.clientX - rect.left;
  var y = event.clientY - rect.top;

  document.getElementById('x').value = x;
  document.getElementById('y').value = y;
});
```

3. Server-Side Processing:

Upon submission of the form, the coordinates are sent to the server, where they are processed using a server-side scripting language (such as PHP, Python, or Node.js). The server compares the received coordinates with predefined areas of interest and performs actions based on the region clicked, such as redirection or dynamic content generation.

Example (PHP script to process coordinates):

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $x = $_POST['x'];
    $y = $_POST['y'];

    // Define the clickable regions with their coordinates (rectangular regions)
    if ($x >= 34 && $x <= 270 && $y >= 44 && $y <= 350) {
        // USA region clicked
        header("Location: https://www.usa.gov");
    } elseif ($x >= 400 && $x <= 450 && $y >= 200 && $y <= 250) {
        // Brazil region clicked
        header("Location: https://www.brazil.gov.br");
    } else {
        echo "Region not found.";
    }
}
?>
```

. Advantages

- 1) **Dynamic Content:** Allows for complex, dynamic interactions, such as database queries and content retrieval.
- 2) **Greater Flexibility:** Can handle more advanced tasks than client-side solutions, such as user-specific actions.
- 3) **Security:** Sensitive operations are processed on the server, offering better control and security.
- 4) **Support for Complex Logic:** Enables handling of complex business logic and multi-step processes.

Limitations

- 1) **Slower Response:** Requires server communication, causing delays compared to client-side solutions.
- 2) **Increased Complexity:** Requires both client-side and server-side programming (e.g., PHP, Python).
- 3) **SEO Issues:** Search engines may not index clickable areas effectively.
- 4) **Server Dependency:** Performance depends on server availability and reliability.

Use Cases

- 1) **Interactive Maps:** Clicking regions of a map to fetch region-specific data or redirect to other pages.
- 2) **Product Images:** Clicking different parts of a product image to show configurations or specifications.
- 3) **Data Visualizations:** Interactive diagrams where clicking on regions triggers dynamic updates from the server.

Best Practices

- 1) **Accurate Coordinates:** Define precise coordinates and ensure they work across devices.
- 2) **Descriptive alt Text:** Enhance accessibility with meaningful alt attributes.
- 3) **Validate Coordinates:** Ensure correct region identification and prevent malicious inputs.
- 4) **Optimize Performance:** Use techniques like caching to reduce server load and response time.
- 5) **Test Responsiveness:** Ensure the imagemap functions across different screen sizes.

Client-Side vs. Server-Side Imagemaps

Client-Side Imagemaps	Server-Side Imagemaps
Processing occurs directly within the user's browser	Coordinates are transmitted to the server for processing and response generation
Immediate response time due to the absence of server communication	Response time is slower due to the necessity of server interaction and round-trip data transfer
Implementation involves basic HTML and JavaScript for defining regions and handling clicks	Requires both client-side (HTML, JavaScript) and server-side (e.g., PHP, Python, Node.js) scripting, making it more complex
Primarily limited to static interactions, with predefined regions and links	Supports dynamic interactions, including database queries, content generation, and real-time updates
Security is lower as processing is conducted on the client side, making it more vulnerable to exploitation	Enhanced security, as sensitive operations and data access are controlled server-side, reducing exposure to malicious activities
Accessibility depends on the client's browser capabilities, which may vary across different devices and environments	More reliable in terms of accessibility since processing does not rely on the client's device and browser configuration
Well-suited for simple, static image maps, diagrams, and basic navigation elements	Ideal for more complex applications requiring dynamic content, user-specific interactions, and real-time data retrieval
Once the image is loaded, it operates independently of the server, minimizing external dependencies	Dependent on server availability and performance, making the application vulnerable to server downtimes or slowdowns

Unit -2 HTML

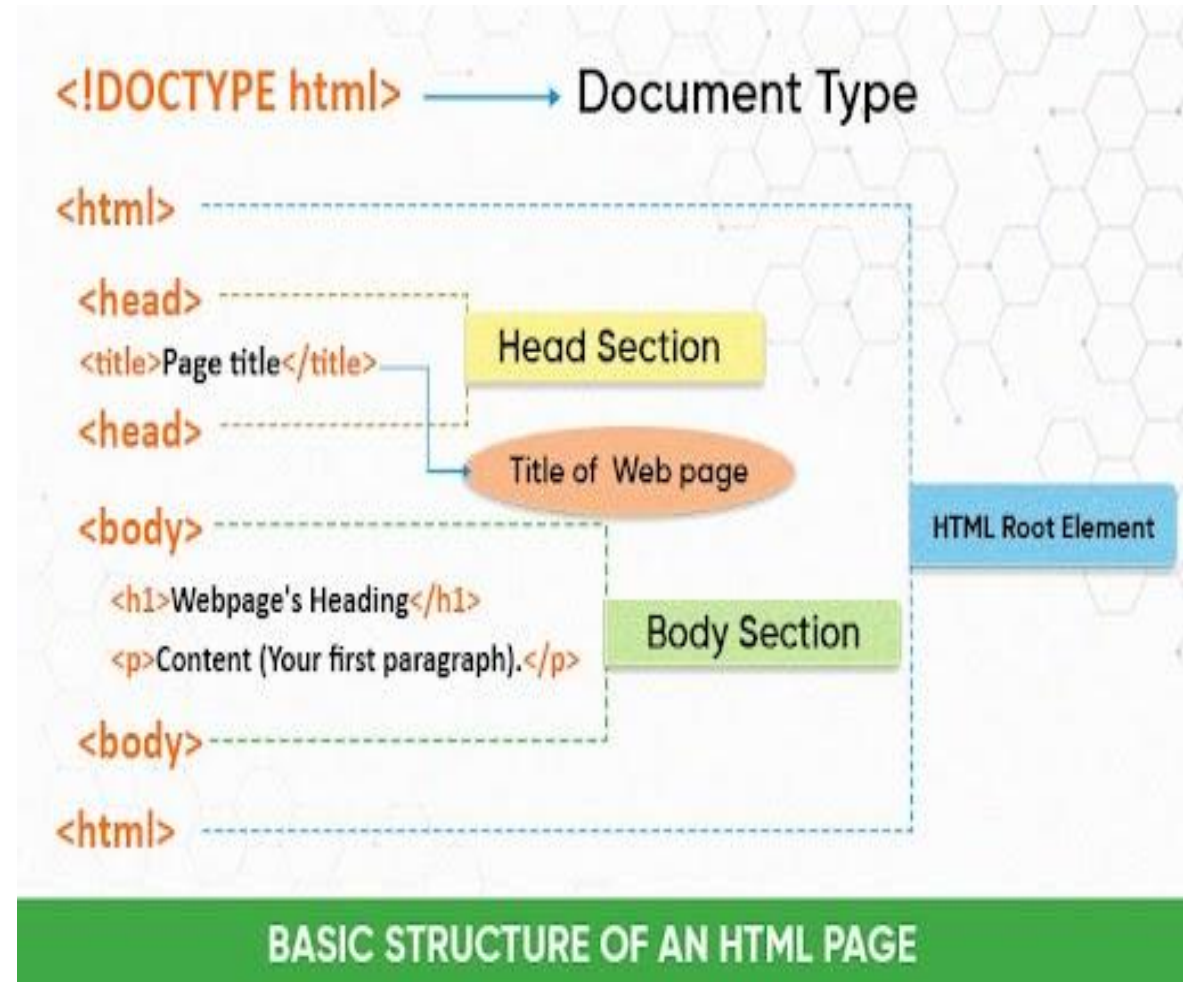
- HTML stands for HyperText Markup Language.
- It is used to structure the content on the web by using various elements (commonly known as tags).
- These HTML elements define the different sections of a web page, such as headings, paragraphs, links to other webpages, listings, images, tables, etc. These elements tell the browser about the content and formatting to display.
- HTML is HyperText + Markup Language. Where,
- **HyperText** refers to the way in which Web pages (HTML documents) are linked together. Thus, the link available on a webpage is called "HyperText".
- **Markup Language**, which means you use HTML to simply "mark up" a text document with tags that tell a Web browser how to structure it to display.

Why is HTML Important?

- **Foundation of Web Pages:** HTML provides the basic structure for all web content, defining headings, paragraphs, images, and links.
- **Organizes Content:** It organizes content in a clear and logical manner, making it easy for users to navigate and understand.
- **Enhances User Experience:** HTML works with CSS and JavaScript to improve design and interactivity, ensuring a better user experience.
- **Search Engine Optimization (SEO):** Proper HTML structure helps search engines index content, improving a page's visibility and ranking.
- **Accessibility:** HTML ensures content is accessible to all users, including those with disabilities, through features like alt text and semantic tags.
- **Responsive Design:** HTML5 supports mobile-friendly, responsive design, allowing websites to adapt to different devices.
- **Cross-Browser Compatibility:** HTML ensures that web pages are displayed consistently across various web browsers.
- **Foundation for Other Technologies:** HTML is the starting point for learning other web technologies like CSS and JavaScript.

HTML Page Structure

- HTML page structure (or, HTML basic structure) consists of the essential elements that are required to create an HTML document that can be displayed on the browser.
- HTML page structure contains `<!DOCTYPE html>`, `<html>`, `<head>`, `<title>`, `<body>`, and other tags that will be shown on the homepage. Where,
`<!DOCTYPE html>` – It defines the document type as HTML.
`<html>` – It is a root element for an HTML document. All elements are placed inside this element.
`<head>` – The **head** tag may contain scripts and styles that are useful page functionalities. Meta tags are also placed inside this tag.
`<title>` – It defines the webpage's title.
`<body>` – It defines the body of the webpage, all elements that are used to display content on the browser placed inside the **body** tag.



History of HTML

Version	Release Year	Description	Key Features
HTML 1.0	1991	The first version of HTML, developed by Tim Berners-Lee at CERN. It was designed to enable basic text-based documents and links on the web.	- Basic structure for text-based documents. - - Only essential tags (headings, paragraphs, links).
HTML 2.0	1995	First standardized version of HTML by the IETF (Internet Engineering Task Force). It provided the foundation for more structured and interactive websites.	- First standardized version by the IETF. - Introduced tables and forms. - Improved document structure.
HTML 3.2	1997	HTML 3.2 added more advanced features like scripting, JavaScript support, and improved multimedia handling. This version supported more interactivity.	- Added support for JavaScript. - Enhanced tables and applets. - Improved interactivity.

Version	Release Year	Description	Key Features
HTML 4.0	1997	➤ HTML 4.0 introduced a major update with the separation of content and presentation (HTML and CSS). ➤ It also focused on accessibility and support for multiple languages.	- Separated content (HTML) from presentation (CSS). - Introduced frames for multiple web pages. - Improved accessibility and internationalization.
XHTML	Early 2000s	➤ XHTML was introduced as a stricter version of HTML, following XML syntax rules. ➤ It aimed for cleaner code and more consistent display across browsers, though it had compatibility issues.	- Stricter version of HTML, following XML syntax. - Aimed at cleaner code and better browser compatibility, but not widely adopted.
HTML5	2014	➤ HTML5 is the latest version and brought major improvements, including multimedia support, new elements, APIs, and better mobile responsiveness. ➤ It is widely used today for modern web applications.	- Native multimedia support (<audio>, <video>) - New semantic elements (<article>, <section>, <nav>). - Improved forms with new input types. - JavaScript APIs for local storage, geolocation, and offline support. - Supports responsive design for mobile and desktop devices.

HTML Tags

- HTML tags are keywords enclosed in angle brackets (< >) that describe the content contained within them.
- Tags define the structure and presentation of the content in a webpage.
- Tags can be classified into start tags, end tags, and self-closing tags.
 - **Start Tag:** Marks the beginning of an element (e.g., <p>).
 - **End Tag:** Marks the end of an element (e.g., </p>).
 - **Self-closing Tag:** These tags do not require an end tag. For example, the image tag does not have a closing tag.

What is an HTML Element?

- An **HTML element** is a fundamental building block used to create a webpage.
- It is made up of a **start tag**, the **content** itself, and an **end tag**.
- The element defines what type of content is being displayed and how it should be structured or formatted.

➤ The basic syntax of an HTML element is:

<tag_name>content</tag_name>

❖ **<tag_name>**: This is the start tag, which indicates the beginning of the element.

❖ **content**: The actual content placed between the start and end tags. This can be text, images, links, etc.

❖ **</tag_name>**: This is the end tag, marking the end of the element.

➤ **Example** : `<p>This is a paragraph of text.</p>`

HTML Attributes

➤ An **attribute** in HTML is a property that can be added to an HTML tag to define or modify the behavior or appearance of an element.

➤ Every HTML element can have one or more attributes, and these attributes give more information about that element or control its properties.

➤ They are placed inside the start tag and usually come in name-value pairs (e.g., `name="value"`).

➤ **Common Attributes**: `class`, `id`, `href`, `src`, `alt`, `style`, `target`

➤ **Syntax**: The attribute name is followed by an equal sign (=) and the attribute value is enclosed in quotes (" ").

➤ **Example**: ``

- `src`: Specifies the image source.
- `alt`: Provides alternative text for the image.
- `width` and `height`: Define the size of the image.

Global Attributes in HTML

- In HTML, **global attributes** are special attributes that can be applied to most HTML elements.
- These attributes offer functionality that is not tied to a specific type of element but can be used universally across multiple tags.
- They play a crucial role in enhancing the structure, accessibility, and interactivity of web pages.

Attribute	Description	Example
id	Provides a unique identifier for an element.	<div id="header">Header Content</div>
class	Assigns one or more class names to an element for styling or scripting.	<p class="intro">This is an intro paragraph.</p>
style	Allows inline CSS to be applied directly to an element.	This is a blue heading
title	Provides additional information about an element, shown as a tooltip.	Visit Example
lang	Specifies the language of the element's content.	<html lang="en">This page is in English.</html>

Document Structure Tags

- In HTML, document structure tags are fundamental as they define the organization and layout of the webpage.
- These tags are responsible for creating a proper and well-organized structure for any webpage.

1.<html>

- The <html> tag is the root element of an HTML document.
- It is the outermost container for the entire webpage, encompassing all other HTML elements.
- All HTML content, including the <head> and <body>, must be placed inside the <html> tag.
- The <html> tag may contain the lang attribute to specify the language of the document (e.g., lang="en" for English).

<html lang="en">

<!-- Other tags like head, body, etc. go here -->

</html>

2.<head>: The Metadata Container

- The <head> tag is used to contain meta-information about the document that isn't directly visible on the webpage.
- It holds important elements like the title, links to external CSS files, and metadata such as the character set or author.
- **Common Elements Inside <head>:**
 - <**title**>: Defines the title of the webpage, which is displayed in the browser's title bar or tab.
 - <**meta**>: Provides metadata about the document (e.g., author, character encoding).
 - <**link**>: Links external resources like CSS stylesheets.
 - <**style**>: Contains internal CSS for the page.
 - <**script**>: Contains or links to JavaScript files.

Example:

```
<head>  
  <meta charset="UTF-8">  
  <title>My Webpage</title>  
  <link rel="stylesheet" href="styles.css">  
</head>
```

3.<body>: The Content of the Webpage

- The <body> tag is where the visible content of the webpage is placed.
- Everything that appears on the webpage, such as text, images, links, and other multimedia elements, is included within the <body> tag.
- All content that is meant to be displayed to the user is enclosed in the <body> tag.

➤ **Example:**

```
<body>  
  <h1>Welcome to My Webpage!</h1>  
  <p>This is a paragraph of text.</p>  
</body>
```

<header>: The Header Section

- The <header> tag is used to define the header section of a webpage.
- This can include elements like a website title, navigation links, and introductory content.
- The <header> tag is often used within the <body> tag and can be placed at the top of the webpage or within specific sections.
- Elements Commonly Found in <header>:
 - <h1> to <h6>: Headings that describe the content.
 - <nav>: Navigation links.

➤ Example:

```
<header>
<h1>My Website</h1>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
```

<footer>: The Footer Section

- The <footer> tag is used to define the footer content of the webpage.
- This can include copyright information, contact links, or site navigation.
- The <footer> is placed at the bottom of the document or within a specific section of the page.
- **Example:**

```
<footer>  
  <p>&copy; 2025 My Website</p>  
  <a href="#privacy-policy">Privacy Policy</a>  
</footer>
```

<section>: Defines a Section of Content

- The <section> tag is used to define a thematic section of content on the webpage. Each section can represent a distinct part of the webpage, such as a chapter, a group of related content, or a module.
- The <section> tag should contain headings and content related to a specific topic.

Example:

```
<section>  
  <h2>About Us</h2>  
  <p>This is some content about the company.</p>  
</section>
```

<article>: Represents Independent Content

- The <article> tag is used to represent content that is self-contained and can be reused or distributed independently (e.g., blog posts, news articles).
- The <article> tag is useful when the content is standalone and can make sense on its own, even outside of the current webpage.
- Example:

```
<article>
  <h2>Article Title</h2>
  <p>This is an article about web development.</p>
</article>
```

<nav>: Defines Navigation Links

- The <nav> tag is used to group together navigation links, such as a menu or set of links that lead to other parts of the website.
- The <nav> element is usually placed within the header or footer but can be placed anywhere on the page.
- Example:

```
<nav>
  <a href="#home">Home</a> |
  <a href="#about">About</a> |
  <a href="#services">Services</a>
</nav>
```

Formatting Tags in HTML

- **Formatting tags** in HTML are crucial for altering the visual appearance and organizing content on a webpage.
- These tags are primarily responsible for how text and elements are presented to the user and contribute to the overall structure of the document.
- Formatting tags can be classified into two main categories:
 1. **Text-level Formatting Tags**
 2. **Block-level Formatting Tags**
- Both categories serve different roles in the document's structure and presentation.
- Understanding when and how to use each is essential for creating well-structured and visually appealing webpages.

1. Text-level Formatting Tags

- **Text-level formatting tags** are used to style **inline text** within a block of content, such as a paragraph, without affecting the flow or structure of the document.
- These tags modify the **appearance** or **emphasis** of individual text elements, allowing for fine-grained control over the styling of text content.

Characteristics of Text-level Formatting Tags:

- They do not break the flow of the content. They only affect the text they wrap around.
- These tags apply styles to smaller portions of content (words or phrases).
- Primarily affects the **appearance** of the text (e.g., bold, italic, underline) or its semantic meaning (e.g., strong emphasis, emphasis).

Tag	Description	Example
	This tag is used to render the enclosed text in bold, typically for stylistic emphasis. It does not carry any semantic meaning.	This is bold text.
<i>	This tag renders the enclosed text in italic, typically used for stylistic emphasis or to highlight technical terms.	This is <i>italicized</i> text.
<u>	This tag underlines the enclosed text. It is used to convey emphasis, although its use in modern web design is minimal.	This is <u>underlined</u> text.
	This tag represents emphasized text, often displayed in italic. It conveys the meaning of emphasis, suggesting the text is of greater importance.	This is important text.
	This tag indicates strong emphasis, typically rendered in bold. It carries semantic meaning, suggesting that the enclosed content is of significant importance.	This is important text.
<mark>	This tag highlights the enclosed text, often with a yellow background, typically used to mark text that is of relevance.	This is <mark>highlighted</mark> text.
<small>	This tag reduces the font size of the enclosed text, typically used for less important information.	This is small text.
	This tag represents deleted text, often displayed with a strikethrough to indicate that it has been removed or is no longer valid.	This is removed text.
<ins>	This tag represents inserted text, often rendered with an underline, indicating new additions to content.	This is <ins>added</ins> text.
<sub>	This tag renders the enclosed text as subscript, typically used in mathematical formulas and chemical compounds.	H₂O is water.
<sup>	This tag renders the enclosed text as superscript, often used in mathematical equations or footnotes.	E = mc²

2. Block-level Formatting Tags

- **Block-level formatting tags** are used to organize and structure larger sections of content.
- These tags create **block-level elements** that **occupy the full width** available within their parent container and generally appear on **new lines**.
- They are typically used for larger structural elements, such as paragraphs, divisions, headings, or lists.

Characteristics of Block-level Formatting Tags:

- These tags **occupy full width** and create a **new line** before and after the element.
- They are used for grouping content together, such as defining sections of a page, grouping headings, or separating paragraphs.
- These tags impact the overall layout and **document structure**, helping organize content logically and semantically.

Tag	Description	Example
<div>	The <div> tag serves as a generic container used for grouping other elements together. It has no intrinsic semantic meaning but is commonly used for layout and styling when paired with CSS.	<div class="container">Content here</div>
<p>	The <p> tag defines a paragraph of text. It is used to group related sentences, enhancing readability by separating textual content into logical blocks.	This is a paragraph of text.
<h1> to <h6>	The <h1> to <h6> tags define headings at various levels, from the most important (<h1>) to the least important (<h6>). These tags help establish a hierarchical structure for the document, enabling both human readers and search engines to understand the document's organization.	Main Heading Subheading
	The tag creates an unordered list, where list items are typically preceded by bullets. It is used when the sequence or order of items is not of primary importance.	- Item 1 - Item 2
	The tag defines an ordered list, where items are numbered sequentially. It is employed when the order of items holds semantic significance.	- First item - Second item
	The tag represents an individual list item within either an ordered () or unordered () list.	- List item 1 - List item 2
<blockquote>	The <blockquote> tag is used to denote a block quotation, typically employed for longer quotations or cited material. It is conventionally indented to differentiate the quoted content from the surrounding text.	<blockquote>This is a blockquote example.</blockquote>
<hr>	The <hr> tag generates a horizontal rule (a line), typically used to create a visual separation between distinct sections of content, thereby improving the document's organization.	<hr>

List Tags in HTML

- In HTML, **list tags** are used to present content in an organized manner, typically as a sequence of related items.
- Lists are essential for presenting information such as categories, steps in a process, or other types of data in a structured format.
- These tags are key for improving readability, structure, and the overall user experience of a webpage.
- HTML provides several types of list tags, each with a specific purpose based on the type of data being presented.

Types of List Tags in HTML

1.Unordered List ():

- The tag is used to define an unordered list, where the order of the list items does not matter.
- By default, items within an unordered list are marked with bullet points.
- Unordered lists are typically used when the sequence of items is not of primary importance.

Syntax:

```
<ul>  
  <li>Item 1</li>  
  <li>Item 2</li>  
  <li>Item 3</li>  
</ul>
```

Example:

```
<ul>  
  <li>Computer Science</li>  
  <li>Electrical Engineering</li>  
  <li>Civil Engineering</li>  
</ul>
```

- Computer Science
- Electrical Engineering
- Civil Engineering

2.Ordered List ():

- The tag defines an ordered list, where the order of the list items is significant.
- Items within an ordered list are displayed with numerical or alphabetical markers, depending on the specified style.
- Ordered lists are typically used for presenting steps, rankings, or items that need to follow a particular sequence.

Syntax:

```
<ol>  
  <li>Step 1</li>  
  <li>Step 2</li>  
  <li>Step 3</li>  
</ol>
```

Example

```
<ol>  
  <li>Preheat the oven to 180°C.</li>  
  <li>Mix the ingredients.</li>  
  <li>Bake for 30 minutes.</li>  
</ol>
```

1. Preheat the oven to 180°C.
2. Mix the ingredients.
3. Bake for 30 minutes.

the tag creates a numbered list, and each tag represents a step in the recipe process. The items will be listed in numerical order, indicating the sequence of action

List Item ():

- The tag is used to define a list item.
- It is a container element that can be used inside both (unordered list) and (ordered list) tags.
- Each represents an individual entry within the list.

Definition List (<dl>):

- The <dl> tag is used for defining a definition list.
- This type of list is typically used for displaying a series of terms and their corresponding definitions.
- It is composed of a combination of two elements: the <dt> (definition term) tag and the <dd> (definition description) tag.

<dl>

<dt>HTML</dt>

<dd>A markup language used to structure content on the web.</dd>

<dt>CSS</dt>

<dd>A stylesheet language used to style web pages.</dd>

</dl>

HTML

A markup language used to structure content on the web.

CSS

A stylesheet language used to style web pages.

Attributes for List Tags:

type Attribute (for): The type attribute can be used in an ordered list to specify the marker type for the list items.

For example:

"1" for numbers (default).

"A" for uppercase alphabetic markers.

"a" for lowercase alphabetic markers.

"I" for uppercase Roman numerals.

"i" for lowercase Roman numerals.

```
<ol type="A">
```

```
<li>First item</li>
```

```
<li>Second item</li>
```

```
</ol>
```

A. First item

B. Second item

start Attribute (for):

- The start attribute allows you to define the starting number in an ordered list.
- By default, ordered lists start from 1, but this attribute can be used to change the starting value.

➤ Example:

```
<ol start="5">  
  <li>Fifth item</li>  
  <li>Sixth item</li>  
</ol>
```

5. Fifth item
6. Sixth item

type Attribute for (Unordered List)

The type attribute could take the following values:

1. **disc (default):** Represents a solid circle bullet point.
2. **circle:** Represents a hollow circle bullet point.
3. **square:** Represents a square bullet point.

```
<!-- Disc (default) -->  
<ul type="disc">  
  <li>Item 1</li>  
  <li>Item 2</li>  
</ul>
```

```
<!-- Circle -->  
<ul type="circle">  
  <li>Item 1</li>  
  <li>Item 2</li>  
</ul>
```

```
<!-- Square -->  
<ul type="square">  
  <li>Item 1</li>  
  <li>Item 2</li>  
</ul>
```

- Item 1
- Item 2

- Item 1
- Item 2

- Item 1
- Item 2

Nested Lists

- Nested lists in HTML provide a means of organizing content in a hierarchical structure, where list items can contain additional sublists, facilitating the representation of parent-child relationships.
- Both unordered () and ordered () lists can be nested within each other, offering flexibility in presenting information, whether as bullet points or numbered steps.
- Cascading Style Sheets (CSS) can be applied to nested lists to adjust various visual aspects, including the type of list markers, indentation, and overall layout, thereby enhancing the clarity and hierarchy of the content.
- Nested lists are particularly effective in representing complex data structures, such as categories and subcategories, procedural steps, or hierarchical models like organizational charts.

Nested Unordered List

```
<ul>
  <li>Item 1
    <ul>
      <li>Subitem 1a</li>
      <li>Subitem 1b</li>
    </ul>
  </li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

- Item 1
 - Subitem 1a
 - Subitem 1b
- Item 2
- Item 3

Nested Ordered List

```
<ol>
  <li>Item 1
    <ol>
      <li>Subitem 1a</li>
      <li>Subitem 1b</li>
    </ol>
  </li>
  <li>Item 2</li>
  <li>Item 3</li>
</ol>
```

1. Item 1
 1. Subitem 1a
 2. Subitem 1b
2. Item 2
3. Item 3

Mixed nested list

```
<ul>
  <li>Item 1
    <ol>
      <li>Subitem 1a</li>
      <li>Subitem 1b</li>
    </ol>
  </li>
  <li>Item 2</li>
  <li>Item 3
    <ul>
      <li>Subitem 3a</li>
      <li>Subitem 3b</li>
    </ul>
  </li>
</ul>
```

- Item 1
 1. Subitem 1a
 2. Subitem 1b
- Item 2
- Item 3
 - Subitem 3a
 - Subitem 3b

Hyperlink Tags (<a> Tag)

- The anchor tag (<a>) is one of the most fundamental and widely used elements in HTML (Hypertext Markup Language).
- It serves as the primary means of creating hyperlinks, which enable navigation between web pages, sections within the same page, external resources, email addresses, or downloadable files.
- The <a> tag is essential to the structure of the World Wide Web, allowing users to traverse content across different domains and documents seamlessly.
- In the context of web development, hyperlinks play a pivotal role in enabling interconnectivity between different types of content, creating a rich, navigable web experience.
- By embedding a link within text, images, or other HTML elements, developers can guide users to related resources, enriching the user interface and facilitating information discovery.
- The <a> tag has a straightforward yet versatile syntax and is capable of handling various types of resources, making it a core component of navigation and interactivity on the internet.

The basic syntax for creating a hyperlink is as follows:

`<a href="URL">Link Text</a>`

Where:

href (Hypertext Reference) specifies the destination URL of the link.

Link Text is the clickable text that will be displayed on the webpage.

Example: `<a href="https://www.google.com">Google</a>`

Image

- In HTML, images are embedded using the tag.
- Images help enhance the visual appeal of a webpage, making it more engaging for users.
- The tag can display various types of images, such as photos, graphics, and icons.
- It is one of the most commonly used elements in web development.
- Basic Syntax:

Where :

src: Specifies the path to the image file (local or URL).

alt: Provides alternative text that appears when the image is not visible (important for accessibility).

width and height: Define the size of the image on the page (optional).